DFSha

Sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad

UNIVERSIDAD EAFIT

Proyecto 1 · ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos · 2026-2

Opción 1 · Arquitectura Cliente/Servidor con servidor compuesto y distribuido (S2S)

Equipo 8 y 6

Integrantes: Juan Gonzalo De Los Rios; Andres Echeverri; Samuel Molina

Profesores: Alvaro Ospina y Edwin Montoya

Fecha: 15 de octubre de 2026

Repositorio: [agregar enlace del repositorio]

Objetivo y marco teórico

Objetivo general

Diseñar e implementar un sistema de archivos distribuido (DFS) que permita almacenar y acceder a archivos grandes de forma distribuida en múltiples nodos, considerando alta disponibilidad, rendimiento, transparencia y seguridad, y en el que tanto la escritura como la lectura se reparten entre varios nodos.

Objetivos específicos

1. Definir formalmente el servicio (requisitos funcionales y no funcionales) para la **Opción 1**: arquitectura Cliente/Servidor con un servidor compuesto y distribuido (S2S).
2. Implementar un plano de datos basado en **bloques replicados** y un plano de control **sin punto único de fallo**.
3. Resolver el problema de **consistencia** de los datos replicados con garantías explícitas y verificables.
4. Garantizar **seguridad** en comunicaciones, almacenamiento, acceso y entre nodos.
5. Demostrar con pruebas automáticas y mediciones la **escalabilidad**, la **disponibilidad** y el **rendimiento** del sistema, y desplegarlo de forma nativa, en Docker y en IaaS.

Marco teórico

Sistemas de archivos distribuidos

Un DFS permite compartir y acceder de forma concurrente a archivos almacenados en distintos nodos. **NFS** (Sun, 1985) expone un sistema de archivos remoto al sistema operativo del cliente mediante RPC y ofrece transparencia de acceso total con semántica *close-to-open*; **AFS** (Andrew, CMU) añade caché de archivos completos con *callbacks*; **SMB/CIFS** es el equivalente del mundo Windows. Estos sistemas se integran con el sistema de archivos del SO (**basados en bloques/POSIX**), pero su escalabilidad está limitada por el servidor que mantiene el estado.

GFS (Google, 2003) y **HDFS** (Hadoop, 2010) resuelven la escala con una arquitectura *maestro–trabajadores*: un **NameNode/master** guarda los metadatos (espacio de nombres y ubicación de bloques) y los **DataNodes/chunkservers** almacenan bloques grandes replicados; los clientes consultan al maestro y transfieren los datos directamente con los nodos de almacenamiento. Optimizan archivos grandes de escritura única y lectura múltiple (*WORM*), replican en tubería y usan conocimiento de racks para colocar réplicas. Su talón de Aquiles histórico fue el maestro como punto único de fallo.

Los **sistemas de objetos** (p. ej. AWS S3) distribuyen a nivel de archivo completo, no soportan actualización parcial y se usan mediante SDK/CLI, sin integrarse en el SO. DFSha se sitúa entre ambos mundos: **internamente basado en bloques** (como HDFS), con un cliente por **SDK/CLI** (como los sistemas de objetos), pero que además admite lectura aleatoria, anexado y actualización parcial por *copy-on-write* de bloques inmutables.

Consenso y consistencia

Replicar el estado de un servicio exige acordar un orden único de operaciones. **Paxos** y **Raft** (Ongaro y Ousterhout, 2014) resuelven el consenso tolerando la caída de una minoría de réplicas: Raft descompone el problema en elección de líder, replicación del log y seguridad, y garantiza que las entradas comprometidas nunca se pierdan ni se reordenen. Una máquina de estados determinista alimentada por ese log produce réplicas idénticas, y las lecturas servidas por un líder confirmado son **linealizables** (Herlihy y Wing, 1990). El teorema **CAP** (Brewer; Gilbert y Lynch, 2002) obliga a elegir, ante una partición, entre consistencia y disponibilidad: el plano de control de DFSha es **CP** (rechaza en vez de arriesgar divergencia); el plano de datos obtiene disponibilidad con réplicas inmutables.

Seguridad

- **TLS / mTLS** (RFC 8446): confidencialidad e integridad en tránsito; con certificados de cliente los nodos se autentican mutuamente.
- **Cifrado autenticado AES-256-GCM** (NIST SP 800-38D) para datos en reposo, y **cifrado de sobre** (*envelope encryption*, como en los KMS de la nube): una clave de datos por archivo, envuelta por una clave maestra de la que se derivan las demás con HKDF.
- **Contraseñas con scrypt** (RFC 7914, función de derivación *memory-hard*), **TOTP** (RFC 6238) como segundo factor, **API keys** y listas de control de acceso (ACL).

Principios de diseño aplicados

Separación de plano de control y de datos; inmutabilidad de bloques; verificación de integridad extremo a extremo; idempotencia de las operaciones reintentadas; diseño para el fallo (todo fallo es el caso normal); y automatización de la reparación.

Descripción del servicio y problema abordado

1. Problema y objetivo

Almacenar y recuperar **archivos grandes** de forma distribuida en múltiples nodos, con **alta disponibilidad, rendimiento, transparencia y seguridad**. Tanto la escritura como la lectura deben repartirse entre varios nodos, y cada archivo debe estar a su vez particionado en varios nodos.

DFSha es un sistema de archivos distribuido **basado en bloques** (familia GFS/HDFS) con una arquitectura **Cliente/Servidor (Opción 1)** en la que el servidor *S* no es un único proceso, sino un servicio **compuesto y distribuido** (S2S) de dos clases de nodos autónomos que corren dentro de una red privada y exponen APIs abiertas (REST/HTTPS):

Rol	Cantidad típica	Responsabilidad
Cliente DFSha (CLI, shell, SDK Python)	n	Abstracción de sistema de archivos jerárquico tipo Linux; no forma parte del servicio
ControlNode	3 (o 5)	Metadatos, autenticación/autorización, ubicación de bloques, coordinación. Replicados con **Raft**
DataNode	N (≥3)	Almacenan bloques **cifrados**; replican en tubería; verifican integridad

2. Alcance de la Opción 1 (C/S con S compuesto y distribuido)

- **Autonomía S-S**: todos los nodos pertenecen a una sola organización y se autentican entre sí con **mTLS** (CA propia); el servicio corre en una red privada (real o virtual) y solo expone al exterior las APIs de cliente.
- **Mecanismos abiertos C↔S**: HTTPS/JSON (REST) para metadatos y HTTPS binario para bloques; todo nodo expone una API documentada (OpenAPI autogenerada en `/docs` del ControlNode).
- **Composición**: el cliente compone una operación de archivo orquestando ControlNode (metadatos) y DataNodes (datos); los DataNodes componen la tubería de replicación entre sí; los ControlNodes componen el consenso entre sí.

3. Requisitos funcionales

ID	Requisito	Cómo se cubre
RF1	Gestión del sistema de archivos: `ls, cd, mkdir, rmdir, rm, mv, stat, chmod, chown, ACL…`	`dfsha ls/cd/pwd/mkdir/rmdir/rm/mv/cp/cat/tree/chmod/chown/setfacl/getfacl` + `dfsha shell` con directorio de trabajo; SDK `Client.ls/mkdir/…`
RF2	Transferencia de archivos: `send()` / `receive()`	`Client.send/receive` (memoria) y `Client.put/get` (archivos locales); `dfsha put/get`, transferencias **paralelas por bloque**, con progreso y verificación SHA-256
RF3	Acceso a archivos: `open(), close(), read(), write(), lock()…`	`Client.open(path, 'rb'/'wb'/'ab'/'r+b'/texto)` devuelve objetos tipo archivo con `read/seek/write/truncate`; `Client.lock()` (compartido/exclusivo, con *lease* y *fencing token*)
RF4	Autenticación y control de acceso por usuario o grupo	usuario/clave, **2FA TOTP**, API keys, permisos POSIX (rwx dueño/grupo/otros) + **ACL** por usuario/grupo
RF5	Operación administrativa	alta/baja de usuarios y grupos, estado de DataNodes (`active/decommissioning/removed`), `fsck`, `nodes`, métricas Prometheus

4. Requisitos no funcionales (mapeados a la rúbrica)

ID	Requisito	Mecanismo clave	Verificación
RNF1 Escalabilidad	usuarios, archivos, tamaño de archivo	Archivos en bloques inmutables repartidos por política de colocación; plano de datos escala añadiendo DataNodes; metadatos en memoria con lecturas servibles desde seguidores (`--stale`); bloques de tamaño configurable (4 MiB por defecto); asignación de bloques por lotes (1 consenso por ≤512 bloques)	`bench/` (escalado en DataNodes, clientes, tamaño de bloque, paralelismo)
RNF2 Alta disponibilidad	sin caídas del servicio ni pérdida de datos	Raft con 3 réplicas de metadatos (tolera 1 caída; 5 → 2); replicación ×3 de datos; re-replicación automática; *failover* de líder en ≈1,5 s (mediana medida; ≤4 s); el cliente reintenta de forma transparente	pruebas de caos: matar DataNodes/líder a mitad de subida/bajada, caída total del plano de control
RNF3 Consistencia	resolver el reto de consistencia de datos	Metadatos **linealizables** (Raft, lease + check-quorum); bloques **inmutables** y verificados por hash; *commit* atómico de archivos; semántica *close-to-open*; control optimista de versión; locks con fencing	pruebas de Raft (caos aleatorio con invariantes), de la máquina de estados (determinismo) e integración
RNF4 Particionamiento	cada archivo repartido entre nodos	Bloques de tamaño fijo; cada bloque en R nodos de ≥2 zonas; balance por bandas de utilización	`dfsha blocks <archivo>` muestra el mapa de réplicas; test de distribución
RNF5 Rendimiento / concurrencia	accesos concurrentes a archivos y a datos de un archivo	E/S paralela por bloque; tubería de replicación; *read-ahead*; asignación por lotes; *group commit* en el log Raft; lecturas de metadatos con *lease*	benchmarks de throughput y latencia de metadatos
RNF6 Seguridad	cifrado en comunicaciones y en almacenamiento; control de acceso; seguridad entre nodos	TLS 1.2+ en todo; **mTLS** con roles entre nodos; AES-256-GCM por bloque con **cifrado de sobre** (las DataNodes no almacenan las claves); tokens de capacidad sellados; scrypt; 2FA; throttling; auditoría	`tests/test_integration.py` (token binding, mTLS, throttling, 2FA, ACL…)
RNF7 Transparencia	acceso y localización, nunca estática	API tipo archivo; ubicación resuelta dinámicamente con el ControlNode en cada operación; *failover* entre réplicas y líderes invisible; reubicación automática de bloques en escritura	tests de caos
RNF8 Portabilidad / despliegue	nativo o Docker; IaaS (AWS Academy)	Una imagen Docker para todos los roles; `docker-compose.yml` de referencia; guía y scripts AWS	`docs/OPERACION.md`, `deploy/aws/`

5. Decisiones de diseño (resumen)

1. **Basado en bloques con metadatos centralizados-replicados** (estilo HDFS NameNode + DataNodes), pero con el *NameNode* **alta-disponible por consenso** (Raft) en lugar de un respaldo manual.
2. **Bloques inmutables + commit atómico**: simplifica la consistencia (no hay réplicas divergentes de un bloque) y permite reemplazos y *copy-on-write* sin ventanas de lectura inconsistente.
3. **Plano de control y de datos separados**: el consenso solo transporta metadatos; los datos fluyen directamente cliente↔DataNode (el ControlNode nunca es cuello de botella de ancho de banda).
4. **Seguridad por capas**: cada salto de red cifrado y autenticado; cifrado en reposo con claves que no residen en el nodo de almacenamiento.
5. **Reparación continua**: vivacidad por *heartbeat*, informes de bloques, *scrubber* de integridad y re-replicación con periodo de gracia para no mover datos por fallos transitorios.

6. Límites explícitos (fuera de alcance)

- Membresía **estática** de ControlNodes (no hay cambio dinámico de configuración Raft).
- No hay montaje a nivel de SO (FUSE/NFS): la transparencia de acceso se logra mediante SDK/CLI/shell (el enunciado lo admite para sistemas de objetos/bloques con SDK propio).
- El cifrado en reposo protege contra robo de discos de DataNodes; **no** protege contra un compromiso del ControlNode (que custodia las claves envueltas). Ver `SEGURIDAD.md`.
- Un solo dominio administrativo (no federado): la Opción 2 (P2P / SuperPeers) no se implementa.

Arquitectura del sistema y diagramas

1. Vista general

DFSha es un sistema de archivos distribuido **por bloques** con **arquitectura Cliente/Servidor** (Opción 1) donde el servidor es un servicio compuesto: un **plano de control** replicado con Raft y un **plano de datos** de nodos de almacenamiento. Los dos planos están separados a propósito: el consenso sólo transporta metadatos y los bytes de los archivos fluyen directamente entre cliente y DataNodes.

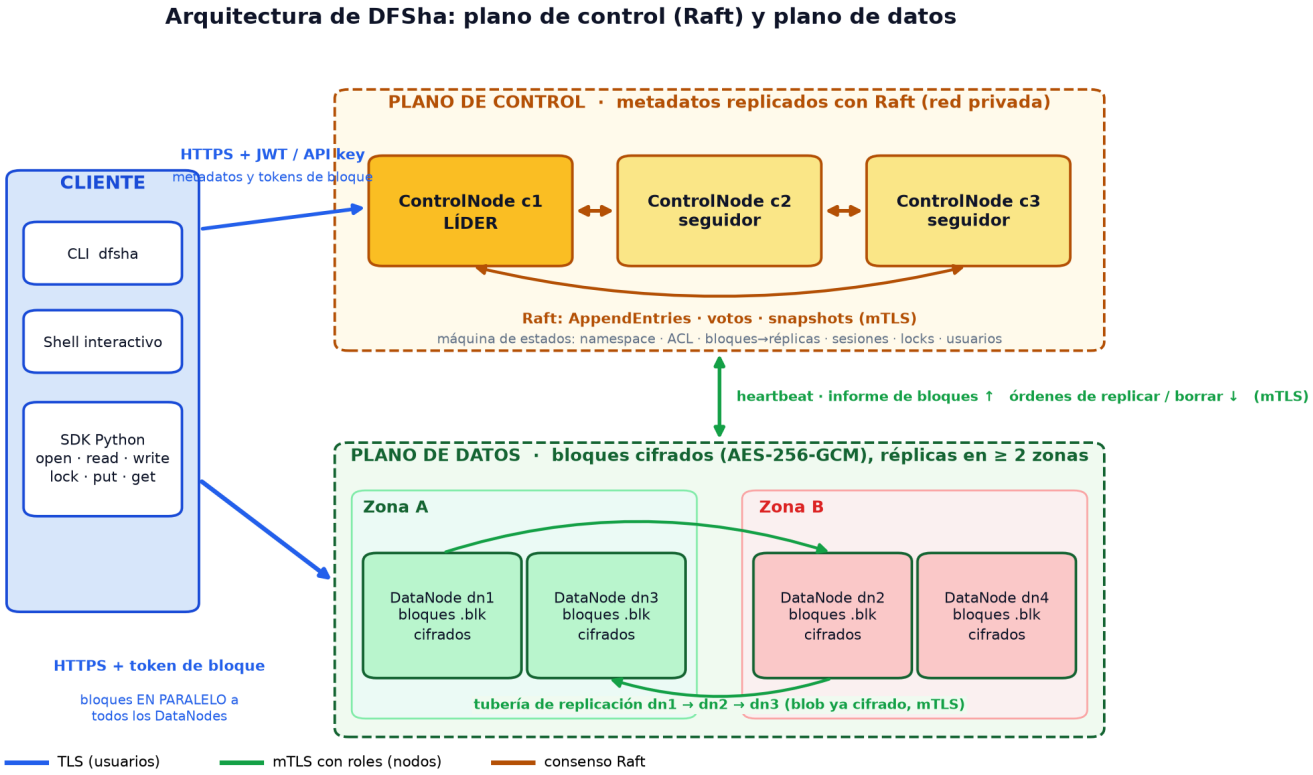


Figura 1. Arquitectura de DFSha: plano de control (Raft), plano de datos y enlaces seguros.

Componentes

Componente	Código	Responsabilidad
Cliente / SDK	`client/sdk.py`, `client/fileobj.py`	Transparencia de acceso, ubicación y fallos; E/S paralela por bloque; verificación SHA-256
CLI y shell	`client/cli.py`	Abstracción jerárquica tipo Linux
ControlNode	`control/node.py`, `api.py`, `monitor.py`	API de metadatos, autenticación, tokens de bloque, coordinación, reparación
Máquina de estados	`control/fsm.py`	Namespace, permisos/ACL, sesiones de escritura, bloques/réplicas, locks, usuarios (determinista)
Raft	`raft/core.py`, `raft/storage.py`	Elección de líder, replicación del log, snapshots, lecturas linealizables
Colocación	`control/placement.py`	Política de réplicas (zonas + balance)
DataNode	`data/node.py`, `store.py`, `api.py`	Bloques cifrados, tubería, informes, *scrubber*
Seguridad	`security/crypto.py`, `auth.py`, `pki.py`	AES-GCM, sobres de claves, tokens, TOTP, scrypt, CA propia

2. Modelo de datos

```
Namespace (replicado por Raft)                Datos (en DataNodes)
──────────────────────────────                ────────────────────
inode ─ dir  : nombre → inode                 bloque = blob cifrado
      └ file : [block_id, ...]  ← layout            MAGIC|ver|gen|nonce|AES-GCM(ct‖tag)|SHA-256
block_id → {len, sha256, réplicas:[dn…], repl}
file → {size, version, repl, block_size, DEK envuelta, owner/group/mode/ACL}
```

- **Bloque**: unidad de distribución, replicación, reparación y verificación. Tamaño por archivo (por defecto 4 MiB; configurable de 64 KiB a 256 MiB). Un bloque puede ser menor que el tamaño máximo (último bloque, anexos).
- **Inmutabilidad**: modificar un archivo = nuevos bloques + nuevo layout (ver `CONSISTENCIA.md`).

3. Flujos principales

3.1 Escritura (`put` / `open('wb')`)

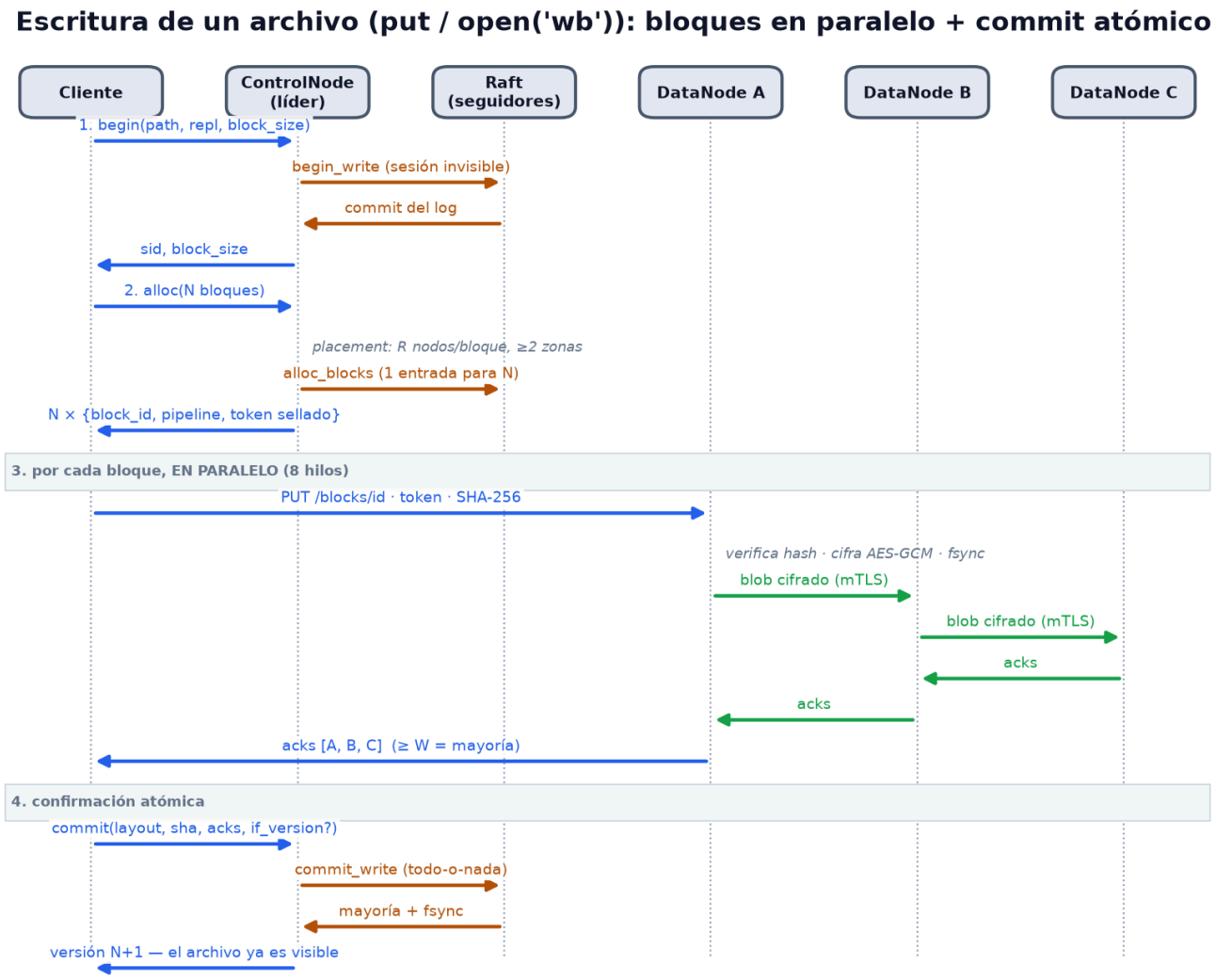


Figura 2. Secuencia de escritura de un archivo: bloques en paralelo, tubería de replicación y commit atómico.

Puntos clave:

- **Asignación por lotes**: una sola entrada Raft reserva hasta 512 bloques; el consenso no está en el camino caliente de los datos.
- **Tubería (como HDFS)**: el cliente sube *una* copia; los DataNodes reenvían. El ancho de banda de subida del cliente no se multiplica por R y el blob viaja ya cifrado (los siguientes nodos jamás ven texto claro ni la clave).
- **Tolerancia a fallos**: si el primer nodo no responde, el cliente entra por la posición siguiente de la tubería; si hay menos de W confirmaciones pide **reubicación** excluyendo los nodos fallidos.

3.2 Lectura (`get` / `open('rb')`)

1. `GET /files/locations`: el líder verifica permisos y devuelve, por bloque, las réplicas **vivas primero** (misma zona primero) y un token de lectura sellado.
2. El cliente descarga los bloques **en paralelo** (o con *read-ahead* en `open`), verifica el SHA-256 del texto claro y, si una réplica falla o está corrupta, **conmuta a otra** de forma transparente (refrescando ubicaciones y tokens si hace falta).

3.3 Elección de líder y *failover*

Si el líder cae, un seguidor vence su temporizador aleatorio, hace **Pre-Vote**, gana la elección y registra una entrada *no-op* que le permite comprometer el término. El cliente recibe `ENOTLEADER` (con pista del nuevo líder) o un error de conexión y reintenta con **el mismo** `X-Request-Id`, de modo que una operación ya comprometida no se ejecute dos veces.

3.4 Autorreparación

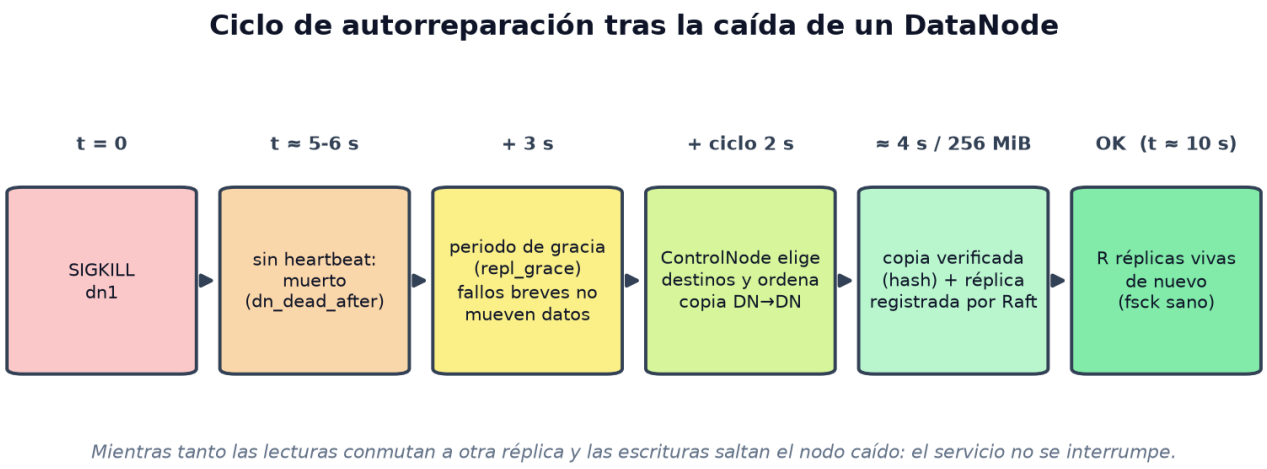


Figura 3. Ciclo de autorreparación tras la caída de un DataNode.

4. Escalabilidad por diseño

Dimensión	Mecanismo
Tamaño de archivo	Bloques; el archivo crece añadiendo bloques (anexar/COW). Sin límite impuesto por un nodo
Nº de archivos	Metadatos en memoria (RAM del ControlNode) + snapshots/compactación del log; **medido**: ≈0,8 KiB por archivo + ≈0,5 KiB por bloque (≈0,01 % del dato con bloques de 4 MiB)
Ancho de banda	Los datos no pasan por el plano de control; E/S **paralela** por bloque entre muchos DataNodes; escala con nº de nodos
Capacidad	Añadir un DataNode: se registra solo, recibe nuevas escrituras y el **balanceador** migra réplicas desde nodos llenos sin reducir redundancia
Lectura de metadatos	Lecturas linealizables con *lease* sin RPC; lecturas relajadas servibles por cualquier ControlNode
Escritura de metadatos	*Group commit* (un `fsync` por lote de entradas) y asignación de bloques por lotes
Usuarios concurrentes	Servidores asíncronos (asyncio/uvicorn); E/S de disco y criptografía en hilos; semáforos de carga por DataNode

La escalabilidad **del plano de control** (cuántos metadatos caben y cuántas operaciones/s) está limitada por un único grupo Raft; es el mismo compromiso de HDFS/GFS (la federación o el *sharding* del namespace sería el siguiente paso; ver "Trabajo futuro" en el informe).

5. Estructura del repositorio

```
src/dfsha/
  raft/        core.py (Raft) · storage.py (log, snapshot, meta durables)
  control/     fsm.py · node.py · api.py · monitor.py · placement.py · config.py · main.py
  data/        store.py · node.py · api.py · config.py · main.py
  client/      sdk.py · fileobj.py · cli.py · session.py
  security/    crypto.py · auth.py
  common/      errors.py · net.py (mTLS/uvicorn) · util.py
  pki.py · cluster.py
tests/         unitarios, integración y caos
bench/         benchmarks reproducibles
docker/ · docker-compose.yml · deploy/aws/
docs/
```

Un archivo de 8 bloques: cada bloque en 3 DataNodes distintos y en ambas zonas

	dn1 zona A	dn2 zona B	dn3 zona A	dn4 zona B	dn5 zona A
bloque 0	R1	R2			R3
bloque 1		R1	R2	R3	
bloque 2	R1		R3	R2	
bloque 3		R2		R3	R1
bloque 4	R3		R1	R2	
bloque 5		R1		R3	R2
bloque 6	R1	R2	R3		
bloque 7		R3		R1	R2

Figura 4. Mapa de réplicas de un archivo de 8 bloques en 5 DataNodes y 2 zonas

Especificación de protocolos y APIs

Todos los nodos exponen una API HTTPS/JSON (REST). El ControlNode publica además su esquema OpenAPI interactivo en `https://<controlnode>:<puerto-api>/docs`.

Enlace	Protocolo	Puerto	Autenticación	Contenido
Cliente ↔ ControlNode	HTTPS REST/JSON	`api` (p. ej. 8441)	`Authorization: Bearer <JWT>` o `X-API-Key`	metadatos, autenticación, asignación y ubicación de bloques, locks
Cliente ↔ DataNode	HTTPS binario	`api` (p. ej. 7441)	`Authorization: Bearer <token de bloque>`	bloques
ControlNode ↔ ControlNode	HTTPS **mTLS** JSON (Raft)	`interno` (9441)	certificado rol `controlnode`	`pre_vote`, `request_vote`, `append_entries`, `install_snapshot`
ControlNode ↔ DataNode	HTTPS **mTLS** JSON	`interno`	certificado rol `datanode` / `controlnode`	registro, heartbeat, informe de bloques, órdenes de replicar/borrar
DataNode ↔ DataNode	HTTPS **mTLS** binario	`interno`	certificado rol `datanode` + token de bloque	tubería de escritura, copia para re-replicación

Por qué REST/HTTPS y no gRPC/MOM: el enunciado lo admite explícitamente; REST es depurable con `curl`, FastAPI genera OpenAPI y validación automática, y el cuello de botella real (copiar bloques de varios MiB) no se beneficia de gRPC. No se usa un MOM porque la coordinación necesaria es síncrona y con confirmación (quórum), no un flujo de eventos desacoplado.

Convenciones

- **Errores**: `{"error": {"code": "EACCES", "message": "...", "extra": {...}}}` con estado HTTP coherente. Códigos estables: `ENOENT` (404) `EEXIST` (409) `ENOTDIR`/`EISDIR`/`EINVAL` (400) `ENOTEMPTY` (409) `EACCES` (403) `EAUTH` (401) `ELOCKED` (423) `ECONFLICT` (412) `ETHROTTLED` (429) `ENOSPC` (507) `EUNAVAIL` (503) `ENOTLEADER` (503, `extra.leader` = URL del líder) `ECORRUPT` (422) `ESTALE` (410).
- **Idempotencia**: las mutaciones aceptan `X-Request-Id`; reenviarlo devuelve el resultado original (también el error original) sin re-ejecutar.
- **Lecturas relajadas**: `?stale=1` permite servir la consulta desde cualquier ControlNode.
- **Paginación**: `GET /v1/files/locations?start=&limit=` (≤1024 bloques por página).
- **Versionado**: prefijo `/v1`.

ControlNode — API pública

Sistema

Método y ruta	Descripción
`GET /health`	sin auth: `{ok, id, role, term, leader}`
`GET /v1/node`	sin auth: rol Raft, término, índices de commit/aplicado
`GET /v1/cluster`	estado completo: ControlNodes, DataNodes, df, fsck, Raft, contadores de mantenimiento

Autenticación

Método y ruta	Cuerpo	Respuesta
`POST /v1/auth/login`	`{username, password, otp?}`	`{token, expires_at, user, groups, admin}` · `EAUTH` con `extra.otp_required` si falta 2FA
`POST /v1/auth/refresh`	–	token nuevo
`GET /v1/whoami`	–	`{name, groups, admin, enabled, totp}`
`POST /v1/auth/password`	`{old, new}`	invalida sesiones previas
`POST /v1/auth/totp/enable` · `/disable`	`{password[, otp]}`	`{secret, uri}` al activar
`POST /v1/auth/apikeys` · `GET` · `DELETE /{id}`	`{label}`	`{id, key}` (la clave sólo se muestra una vez)

Sistema de archivos (RF1)

Ruta	Parámetros
`GET /v1/fs/stat?path=` · `GET /v1/fs/ls?path=` · `GET /v1/fs/df`	lecturas linealizables (o `stale`)
`POST /v1/fs/mkdir`	`{path, parents, mode}`
`POST /v1/fs/rmdir`	`{path}`
`POST /v1/fs/rm`	`{path, recursive, lock_id?}`
`POST /v1/fs/mv`	`{src, dst}` (atómico; sobre directorio destino mueve dentro)
`POST /v1/fs/chmod` · `chown` · `setacl` · `setrepl`	`{path, mode}` · `{path, owner?, group?}` · `{path, entries:[{t,n,p}]}` · `{path, repl}`

Acceso y transferencia de archivos (RF2, RF3)

Ruta	Descripción
`POST /v1/files/begin`	`{path, mode: replace|append|update, overwrite, repl?, block_size?, lock_id?}` → `{sid, block_size, min_repl, version, blocks[], session_ttl}`
`POST /v1/files/{sid}/blocks`	`{count, zone?, exclude[]}` → `blocks[{block_id, pipeline[{id,url}], token}]`
`POST /v1/files/{sid}/realloc`	`{block_id, zone?, exclude[]}` → nueva tubería y token (reubicación tras fallo)
`POST /v1/files/{sid}/commit`	`{layout[{id,len,sha,reps[]}|{id}], size, if_version?, lock_id?}` → `{inode, version, size}` (atómico)
`POST /v1/files/{sid}/abort` · `/renew`	cancelar / extender el *lease* de la sesión
`GET /v1/files/locations?path=&start=&limit=&zone=`	`{size, version, block_size, repl, nblocks, blocks[{block_id,len,sha,locs[{id,url}],token}]}`

Locks

`POST /v1/locks/acquire {path, mode: shared|exclusive, ttl, lock_id?}` → `{lock_id, fence, expires}` · `POST /v1/locks/renew` · `POST /v1/locks/release {lock_id, ttl}`.

Administración (rol admin)

`GET|POST /v1/admin/users` · `PATCH|DELETE /v1/admin/users/{name}` · `GET /v1/admin/datanodes` · `POST /v1/admin/datanodes/{id}/state {state: active|decommissioning|removed}` · `POST /v1/admin/datanodes/{id}/scrub` (verificación inmediata de integridad en reposo; devuelve los bloques corruptos puestos en cuarentena) · `GET /v1/admin/fsck`.

ControlNode — API interna (mTLS)

Ruta	Rol exigido	Descripción
`POST /raft/{pre_vote,request_vote,append_entries,install_snapshot}`	`controlnode`	RPC de Raft (ver abajo)
`POST /internal/dn/register`	`datanode` (CN = `dn`)	alta del DataNode; devuelve la clave de tokens de bloque
`POST /internal/dn/heartbeat`	`datanode`	`{dn, used, capacity, nblocks, load}` a **todos** los ControlNodes
`POST /internal/dn/blockreport`	`datanode`	`{dn, blocks:[{id,n}]}` al líder; reconciliación
`POST /internal/dn/corrupt`	`datanode`	réplicas corruptas halladas
`GET /metrics`	cualquier par mTLS	Prometheus

Mensajes Raft (JSON)

- `append_entries`: `{term, leader_id, prev_index, prev_term, entries[{t,i,c}], leader_commit}` → `{term, success, match_index | conflict_index, conflict_term}`
- `request_vote` / `pre_vote`: `{term, candidate_id, last_log_index, last_log_term}` → `{term, granted}`
- `install_snapshot`: `{term, leader_id, last_index, last_term, data(base64)}` → `{term}`

DataNode — API pública (token de bloque)

Ruta	Descripción
`PUT /v1/blocks/{id}`	cuerpo = texto claro del bloque. Cabeceras: `Authorization: Bearer <token w>`, `X-Block-Sha256`, `X-Pipe-Pos` (posición en la tubería, 0 por defecto). Respuesta: `{acks:[dn…]}`
`GET /v1/blocks/{id}?offset=&length=`	texto claro (descifrado en el nodo con la DEK del token) + `X-Block-Sha256`. `offset/length` permiten lecturas parciales
`GET /health`	`{ok, id, zone, registered, blocks, used}`

DataNode — API interna (mTLS)

Ruta	Rol	Descripción
`PUT /internal/blocks/{id}`	`datanode`	siguiente salto de la tubería: cuerpo = blob cifrado; `X-Block-Token`, `X-Pipe-Pos`, `X-Blob-Sha256`
`GET /internal/blocks/{id}/raw`	`datanode`	blob cifrado (para re-replicación)
`POST /internal/replicate`	`controlnode`	`{block_id, sources[]}`: copiar y verificar desde una fuente
`POST /internal/delete`	`controlnode`	`{blocks[]}`: borrar
`POST /internal/scrub`	`controlnode`, `admin`	verificación de integridad inmediata

Tokens

Token	Formato	Vida	Contenido
Sesión de usuario	JWT HS256	8 h	`sub, ver, iat, exp, jti, amr`
Bloque (lectura)	`v1.` + base64url(AES-GCM)	15 min	`{op:"r", bid, dek, uid}`
Bloque (escritura)	ídem	15 min	`{op:"w", bid, dek, pipe[{id,iu}], uid, max, gen}`

Algoritmos de particionamiento y distribución

1. Particionamiento de archivos en bloques

Un archivo de `size` bytes con tamaño de bloque `bs` se parte en `n = ⌈size / bs⌉` bloques consecutivos; el bloque `i` cubre los bytes `[i·bs, min((i+1)·bs, size))`. Cada bloque tiene un identificador aleatorio de 128 bit, independiente del contenido y del offset.

```
archivo (22 MiB, bs = 4 MiB)
  ├─ bloque 0 [0, 4 MiB)       id=5f2c…  sha256=…   réplicas {dn4, dn5, dn2}
  ├─ bloque 1 [4, 8 MiB)       id=0e19…  sha256=…   réplicas {dn2, dn5, dn3}
  ├─ …
  └─ bloque 5 [20, 22 MiB)     id=b496…  (2 MiB)    réplicas {dn2, dn1, dn4}
```

- El **tamaño de bloque** es una propiedad del archivo (64 KiB – 256 MiB; 4 MiB por defecto). Bloques grandes reducen metadatos y viajes al ControlNode; bloques pequeños mejoran el paralelismo en archivos pequeños (ver benchmark de tamaño de bloque).
- El **layout** del archivo es la lista ordenada de ids de bloque. Cambiarlo es una operación atómica (commit). Los bloques nunca cambian.
- Como los ids son aleatorios, **no hay hot spots por contenido**: dos archivos populares no comparten nodo salvo por azar, y un archivo grande se reparte entre todos los nodos.

2. Colocación de réplicas

Para cada bloque se elige un conjunto de `R` DataNodes **vivos, activos y con espacio**:

```
choose(candidatos, R, excluidos, zona_cliente):
    pool ← candidatos \ excluidos (con espacio libre ≥ bs)
    elegidos ← []
    mientras |elegidos| < R y pool ≠ ∅:
        sub ← pool
        si elegidos = ∅ y zona_cliente existe en pool:   sub ← nodos de zona_cliente       # localidad
        sino si |zonas(elegidos)| < 2 y hay ≥2 zonas:    sub ← nodos de zonas aún no usadas  # diversidad
        a, b ← dos nodos al azar de sub
        elegido ← (a si banda(a) < banda(b)) ; (b si banda(b) < banda(a)) ; (al azar si empatan)
        elegidos.añadir(elegido) ; pool.quitar(elegido)
    devolver elegidos
banda(n) = (⌊ utilización(n) / 5 % ⌋ , ⌊ transferencias_en_curso(n) / 8 ⌋)   # ambas discretizadas: «casi igual» empata
```

Propiedades:

1. **Tolerancia a la pérdida de una zona**: con ≥2 zonas, cada bloque tiene réplicas en ≥2.
2. **Balance sin sesgo**: la *potencia de dos opciones* (Mitzenmacher) reduce el desbalance máximo de O(log n / log log n) a O(log log n) con una sola comparación; las **bandas** del 5 % hacen que nodos casi igual de llenos *empaten* y reciban datos por igual (sin ellas, una diferencia del 0,01 % condenaba al nodo marginalmente más lleno a no recibir nada: defecto hallado y corregido con una prueba de integración).
3. **Costo** O(R) por bloque tras filtrar candidatos O(N); la asignación se hace por lotes de hasta 512 bloques por consenso.
4. Un contador local de bytes asignados evita que una ráfaga apile todos los bloques en el mismo nodo.

3. Escritura: tubería de replicación y quórum

El cliente envía **una** copia a la cabeza de la tubería `[A, B, C]`; cada nodo cifra (sólo el primero), persiste y reenvía el blob al siguiente:

- Ancho de banda de subida del cliente: **U** (con abanico directo sería U/R). Cada DataNode reenvía a lo sumo una copia, de modo que el tráfico se reparte en lugar de concentrarse en el cliente.
- **Quórum de escritura** `W = ⌊R/2⌋ + 1` por defecto: el bloque cuenta como escrito cuando ≥ W nodos confirmaron. Con `R = 3, W = 2` se tolera la caída de un nodo *durante* la escritura sin reintentos.
- **Fallos en la tubería**: si el nodo `k+1` no responde, `k` salta al siguiente; si la cabeza no responde, el cliente entra por la posición siguiente; si `|acks| < W`, se pide una **reubicación** excluyendo los nodos fallidos (hasta 5 intentos).
- El resto de réplicas (hasta `R`) lo completa el líder en segundo plano: la escritura **no espera** a las copias tardías.

Durabilidad aproximada: si cada nodo falla independientemente con probabilidad `p` en la ventana de reparación `T`, un bloque se pierde sólo si los `R` nodos que lo contienen fallan en esa ventana: `p^R`. Con `R = 3` y `p = 10⁻²` por ventana resulta ≈ 10^{-6}; la **re-replicación automática reinicia la cuenta** cada vez que se repara, por lo que `T` es el tiempo de detección (≈ `dn_dead_after`) + gracia + copia, no la vida del dato.

4. Lectura

1. El ControlNode devuelve por bloque las réplicas **vivas primero**, las de la zona del cliente primero y en orden aleatorio entre iguales (reparte la carga de lectura entre las `R` réplicas).
2. El cliente descarga hasta `workers` bloques en paralelo (`get`) o con *read-ahead* de 3 bloques (`open`); verifica `SHA-256(bloque) = sha` del metadato.
3. Ante error de conexión, token vencido, corrupción o ausencia, pasa a la siguiente réplica; si agota todas, **refresca** ubicaciones (puede haber re-replicación en curso) y repite (3 rondas).

5. Detección de fallos, re-replicación y balanceo

```
cada repl_interval en el líder (y warm-up completado):
    para cada bloque con réplicas_vivas < R:
        si algún poseedor está caído hace menos de repl_grace:   continuar      # fallo posiblemente transitorio
        destino ← choose(candidatos, R − |vivas|, excluidos = poseedores ∪ en_curso)
        ordenar(destino, "copiar bloque desde {fuentes vivas}")                 # DN→DN, verificado
        al confirmar: Raft.replica_update(+ réplica)
    para cada bloque con réplicas_vivas > R:
        quitar las de nodos más llenos, sin bajar de 2 zonas
cada balance_interval:
    si no hay bloques sub/sobre-replicados:
        hi, lo ← nodo más/menos usado
        si uso(hi) − uso(lo) > max(2·bs, 20 % · uso_medio):
            copiar hasta 8 bloques hi → lo (respetando zonas)    # el recorte retirará la réplica de hi
```

El *warm-up* tras un cambio de líder evita confundir «aún no recibí heartbeats» con «todos murieron». La decisión de borrar físicamente una réplica sólo se toma cuando el bloque sigue teniendo ≥1 réplica confirmada o cuando el propio DataNode informó de su pérdida.

6. Consenso (Raft) y lecturas

- **Elección**: temporizadores aleatorios U[Tmin, Tmax] (1–2 s por defecto; 0,5–1 s en pruebas). La probabilidad de empate se reduce con la aleatoriedad y con Pre-Vote; el *failover* medido tiene mediana de 1,5 s y máximo de 3,9 s (ver `RESULTADOS.md`).
- **Pre-Vote + leader stickiness**: un nodo sólo inicia una elección real si una mayoría diría "sí"; los seguidores ignoran solicitudes de voto mientras oyen al líder. Un nodo aislado no infla el término.
- **Lease de lectura**: el líder atiende lecturas locales mientras la mayoría haya confirmado su liderazgo hace menos de `0,6 · Tmin`. Es seguro porque ningún seguidor votará por otro candidato antes de `Tmin` desde su último contacto con el líder (premisa: deriva de reloj acotada). Sin lease, una ronda de heartbeats (ReadIndex) confirma el liderazgo.
- **Group commit**: las propuestas concurrentes se agrupan en un solo `append` + `fsync` y una ronda de replicación; el rendimiento de escritura de metadatos crece con la concurrencia.
- **Snapshots**: cada `snapshot_threshold` entradas aplicadas se serializa la máquina de estados y se compacta el log; los seguidores muy atrasados reciben `InstallSnapshot`.

7. Control de concurrencia en archivos

Mecanismo	Uso	Garantía
Commit atómico de layout	toda escritura	todo o nada; lectores ven la versión anterior o la nueva
Versión + `if_version` (CAS)	actualizaciones concurrentes	el segundo escritor falla con `ECONFLICT` en vez de pisar
`append` / `update` con versión base	`ab`, `r+b`	detecta cualquier commit intercalado
Locks compartido/exclusivo	exclusión mutua cooperativa y obligatoria	*lease* con renovación; el servidor rechaza escritores sin el lock
Fencing token	cada adquisición incrementa un contador global	un cliente "zombi" (pausado, lease vencido) se identifica por su token menor

Replicación y consistencia de datos

DFSha separa dos problemas con garantías distintas: la consistencia de los **metadatos** (qué archivos existen, qué bloques los componen, dónde están) y la de los **datos** (el contenido de los bloques).

1. Metadatos: consenso con Raft (CP)

Los N ControlNodes (3 por defecto) mantienen una **máquina de estados replicada** (`control/fsm.py`) mediante **Raft** (`raft/core.py`, implementación propia verificada con pruebas de caos).

Propiedad	Garantía
Escrituras de metadatos	**Linealizables**: pasan por el líder, se replican a una mayoría y se persisten (fsync) antes de confirmarse
Lecturas de metadatos (por defecto)	**Linealizables**: sólo las sirve el líder, con *lease* de líder basado en acks de mayoría (válido porque los seguidores rechazan votos mientras oyen al líder) y *fallback* a ronda de heartbeats
Lecturas relajadas (`--stale`)	Cualquier ControlNode; pueden ir atrasadas unos ms; ante cualquier error se reconfirma con el líder
Tolerancia a fallos	3 nodos tolera 1 caída; 5 nodos tolera 2. Sin mayoría el servicio **rechaza** (Unavailable) en vez de arriesgar inconsistencia: es un sistema **CP** en terminología CAP
Reinicios	Término, voto y log se persisten antes de responder; el estado se reconstruye de snapshot + log
Particiones	Una minoría aislada no puede comprometer ni servir lecturas (check-quorum: el líder renuncia) y un nodo aislado no desestabiliza al líder al volver (Pre-Vote + *leader stickiness*)

Determinismo e idempotencia

- `apply(comando)` es **determinista**: el líder inyecta todo lo no determinista (hora, identificadores, claves, ubicaciones elegidas). Una prueba de *fuzzing* aplica 400 comandos aleatorios a dos réplicas independientes y exige estados idénticos byte a byte.
- Cada comando lleva un `req_id` (cabecera `X-Request-Id`). Si el líder cae tras comprometer pero antes de responder, el cliente reintenta con el mismo id y la máquina de estados devuelve **el mismo resultado sin volver a ejecutar** (tabla acotada incluida en los snapshots).
- El tiempo de la máquina de estados es el máximo de las marcas de tiempo vistas (monótono entre líderes); *leases* de locks y sesiones se evalúan con ese reloj, de forma determinista.

2. Datos: bloques inmutables

Cada archivo es una lista ordenada de **bloques inmutables** identificados por un UUID. Un bloque nunca se modifica: modificar un archivo crea bloques nuevos. Consecuencias:

- **No hay réplicas divergentes** de un mismo bloque (no existe el problema de reconciliar escrituras concurrentes sobre el mismo bloque).
- La corrección de una réplica se decide solo por **su hash**: el cliente calcula SHA-256 del texto claro de cada bloque, el ControlNode lo guarda en el metadato y el cliente lo verifica al leer (**extremo a extremo**, independiente de qué réplica atendió).
- Reparar = copiar un blob íntegro desde otra réplica.

Escritura: tubería con quórum de escritura

1. `begin` crea una **sesión de escritura** (invisible: el archivo aún no existe/no cambió).
2. `alloc` reserva bloques por lotes: el líder elige R DataNodes por bloque y replica la reserva.
3. El cliente envía cada bloque al primer nodo de la tubería; éste lo cifra, lo persiste y lo reenvía (ya cifrado) al siguiente, y así sucesivamente. La respuesta lista los nodos que **confirmaron**.
4. El bloque cuenta como escrito si ≥ **W** réplicas confirmaron (W = mayoría de R; configurable `all/majority/one`). Si no, el cliente pide **reubicación** excluyendo los nodos fallidos.
5. `commit` instala **atómicamente** el nuevo layout del archivo (todos los bloques o ninguno) y sube su `version`. Los bloques viejos pasan a recolección de basura con periodo de gracia.
6. Las réplicas que falten hasta R las completa el ControlNode en segundo plano.

Si el cliente muere a mitad de camino, la sesión expira (*lease*) y sus bloques se recolectan: nunca queda un archivo a medias visible.

Semántica visible para las aplicaciones

Operación	Semántica
`open('wb')` + `close()`	Reemplazo **atómico** (S3-like). Último escritor gana, salvo `if_version` (CAS)
`open('ab')` / `open('r+b')`	Control **optimista**: el commit falla con `ECONFLICT` si otro confirmó antes; `r+b` hace *copy-on-write* de los bloques afectados
Lectura tras escritura	**Close-to-open**: un lector ve la última versión confirmada al abrir; no ve escrituras parciales
Lector abierto durante una reescritura	Sigue leyendo **su** versión (bloques inmutables) hasta el fin del periodo de gracia del GC (`ESTALE` después)
Locks	Compartido/exclusivo con *lease*, renovación automática y **fencing token** monótono. El servidor **hace cumplir** el lock a las escrituras (un escritor sin el lock recibe `ELOCKED`)

3. Reparación y convergencia

Mecanismo	Disparador	Acción
Vivacidad	sin *heartbeat* > `dn_dead_after`	el nodo se considera muerto; sus réplicas dejan de contar
Re-replicación	bloque con réplicas vivas < R, tras `repl_grace`	el líder ordena la copia DN→DN (verificada) y registra la nueva réplica por Raft
Sobre-replicación	réplicas vivas > R (p. ej. un nodo muerto que regresa)	se recortan donde hay menos espacio
Informe de bloques	arranque del DN y periódico	reconcilia: huérfanos se borran; réplicas perdidas (dos informes consecutivos) se descartan y se re-replican
Scrubber	periódico	verifica el hash de cola de cada blob **sin necesitar la clave**; corruptos van a cuarentena y se reportan
Lectura corrupta	fallo de autenticación AES-GCM o hash	el DN pone el blob en cuarentena y avisa; el cliente cambia de réplica
GC	borrado/reemplazo/sesión expirada	cola `pending_deletes` con periodo de gracia; el DN confirma y Raft la vacía

4. Tabla de fallos y comportamiento

Fallo	Efecto	Recuperación
Cae un DataNode	lecturas conmutan a otra réplica; escrituras saltan el nodo o se reubican	tras `repl_grace` se restablece R
Cae el líder Raft	pausa de metadatos de ≈1,5 s (mediana medida con temporizadores de 0,5–1 s; máx. 3,9 s; con los valores por defecto de 1–2 s cabe esperar el doble); el cliente reintenta	nuevo líder; sin pérdida de operaciones confirmadas
Caen 2 de 3 ControlNodes	no hay quórum: se rechazan metadatos (CP). Los **bloques ya ubicados siguen siendo legibles** si el cliente tiene tokens vigentes	al volver un nodo se reanuda
Caída total del plano de control	indisponible	al reiniciar, estado íntegro desde disco
Partición	la minoría no sirve	al sanar, el log de la minoría se descarta
Bit rot en un DataNode	detectado por GCM/hash; réplica descartada	re-replicación desde réplica sana
Disco de DN borrado	informe de bloques detecta la ausencia	re-replicación
Cliente muere a mitad de subida	sesión huérfana	expira y se recolecta; el archivo nunca fue visible

Seguridad

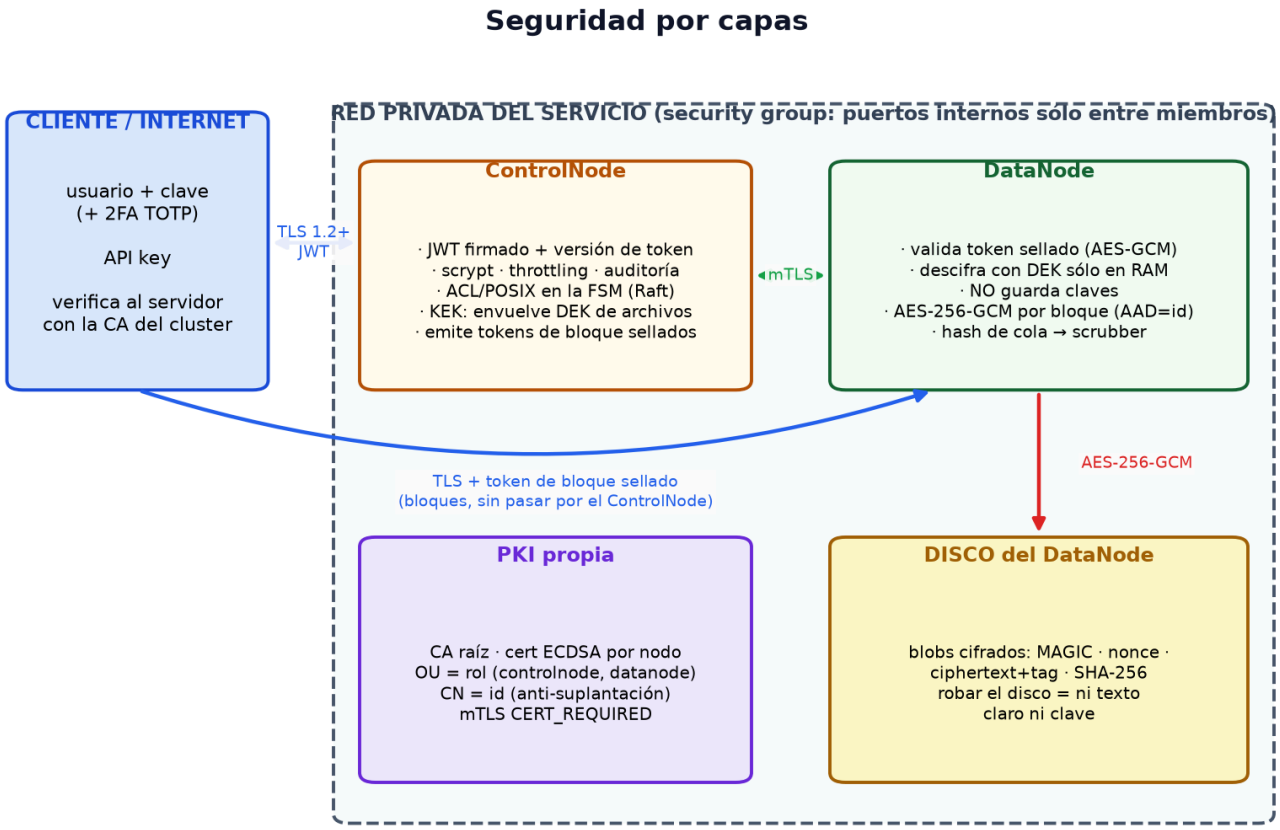


Figura 5. Seguridad por capas: TLS/mTLS, tokens de capacidad, cifrado en reposo y PKI

1. Modelo de amenazas

Actor	Capacidad asumida	Objetivo defendido
Intruso de red (en Internet o en la red privada)	Observa y modifica tráfico	Confidencialidad e integridad en tránsito; suplantación de nodos
Usuario legítimo malicioso	Credenciales válidas propias	No leer/escribir archivos ajenos; no escalar privilegios
Atacante sin credenciales	Alcanza las APIs públicas	Fuerza bruta, tokens forjados, ids maliciosos, cuerpos gigantes
Ladrón de disco / DataNode comprometido en reposo	Lee el disco de un DataNode	Confidencialidad de los datos en reposo
Nodo no autorizado	Intenta unirse o hablar con el plano de control	Integridad del consenso y de los metadatos

Fuera de alcance: un ControlNode comprometido *activamente* (custodia las claves envueltas) y ataques físicos a la memoria de un nodo en ejecución.

2. Cifrado en las comunicaciones

- **Todo** el tráfico es TLS ≥ 1.2: cliente↔ControlNode, cliente↔DataNode, ControlNode↔ControlNode (Raft), ControlNode↔DataNode y DataNode↔DataNode. Ningún puerto atiende HTTP en claro (verificado por test).
- **PKI propia** (`pki.py`): una CA raíz privada firma un certificado ECDSA P-256 por nodo. El campo `OU` del certificado codifica el **rol** (`controlnode`, `datanode`, `admin`) y `CN` el id del nodo.
- **Segmentación**: cada nodo abre dos listeners:
- **público** (TLS de servidor): lo usan los clientes; autentican con JWT/API key o token de bloque.
- **interno** (**mTLS**, `CERT_REQUIRED`): sólo nodos con certificado de la CA. Además, cada ruta interna exige un **rol** concreto (`/raft/*` sólo `controlnode`; `/internal/dn/*` sólo `datanode` y el CN debe coincidir con el `id` declarado, así un DataNode no puede suplantar a otro). En producción el puerto interno se restringe además por *security group*/firewall a la red privada.
- La identidad se extrae del certificado verificado en el *handshake* (`common/net.py`, protocolo uvicorn propio que registra `(CN, OU)` por conexión).

Pruebas: sin certificado → handshake rechazado; certificado válido de rol equivocado → 403; certificado de otra CA → rechazado; DataNode que se hace pasar por otro → 403.

3. Cifrado en el almacenamiento (envelope encryption)

```
cluster.key (256 bit, sólo ControlNodes)
   ├─ HKDF("kek")         → KEK : envuelve la DEK de cada archivo (AES-GCM)
   ├─ HKDF("jwt")         → firma de sesiones (HS256)
   ├─ HKDF("block-token") → sella los tokens de bloque (AES-GCM); también se entrega, por mTLS, a los DataNodes
   └─ HKDF("secrets")     → envuelve secretos TOTP

DEK (256 bit aleatoria por archivo) ── sólo existe envuelta en la máquina de estados (Raft) ──
```

- Cada bloque se cifra con **AES-256-GCM** y un *nonce* aleatorio de 96 bit. El AAD liga el texto cifrado a `(block_id, generación)`: copiar un blob sobre otro bloque falla la autenticación.
- **Los DataNodes no guardan la clave.** La DEK llega *dentro del token sellado* de cada petición y vive en memoria durante la operación. Robar el disco de un DataNode no revela clave ni texto claro (verificado por test: no aparece texto en claro en `blocks/*.blk`).
- Los blobs son **autoverificables sin clave**: una cola SHA-256 permite al *scrubber* detectar *bit rot* en reposo sin descifrar.
- La tubería de replicación reenvía el blob **ya cifrado**: los nodos intermedios ni ven el texto claro ni necesitan la DEK.

4. Autenticación y control de acceso

Usuarios

- **Usuario/clave** con `scrypt` ($N=2^{15}$, r=8, p=1; memory-hard), con tiempo de verificación igualado para usuarios inexistentes (no se filtra qué usuarios existen).
- **2FA TOTP** (RFC 6238; compatible con Google Authenticator/Authy). El secreto se guarda envuelto. Se impide el *replay* (cada *step* sólo se acepta una vez, registrado por Raft) y se prueba contra los vectores del RFC.
- **API keys** (`dfsha_ak_<id>.<secreto>`) para automatización: sólo se almacena el SHA-256 del secreto; revocables.
- **Sesiones** JWT (HS256) con versión de token por usuario: cambiar la clave, deshabilitar o activar 2FA **invalida** de inmediato todas las sesiones previas.
- **Anti fuerza bruta**: bloqueo exponencial tras 5 fallos por usuario (y 50 por IP); aplica también a códigos 2FA erróneos.

Autorización

- Permisos **POSIX** (dueño/grupo/otros, `rwx`) y **ACL** por usuario o grupo (`setfacl`). Para llegar a un archivo se exige `x` en cada directorio del camino; crear/borrar exige `w`+`x` en el padre.
- Rol `admin`: gestión de usuarios y DataNodes.
- Todas las comprobaciones viven en la máquina de estados (se aplican de forma idéntica en todas las réplicas y sin condiciones de carrera entre comandos concurrentes).
- `..` en rutas se resuelve léxicamente en el servidor; los `block_id` deben coincidir con `^[0-9a-f]{32}$` (no hay *path traversal* hacia el disco del DataNode).

Acceso a datos: tokens de capacidad

Un cliente sólo puede tocar un bloque si el ControlNode le entregó un **token sellado** (AES-GCM, TTL 15 min) ligado a `(operación, block_id, usuario, DEK, tubería)`. Los DataNodes no consultan ACL: validan el token. Un token de lectura no sirve para escribir, ni para otro bloque; uno falsificado o vencido es rechazado (verificado por test).

5. Auditoría y observabilidad

- `audit.log` (JSON por línea, en cada ControlNode): logins correctos/fallidos, cambios de clave/2FA, altas/bajas de usuarios, creación/revocación de API keys, mutaciones de espacio de nombres, commits, muerte de DataNodes, réplicas corruptas.
- `/metrics` (Prometheus) en el puerto interno.

6. Endurecimiento operativo (checklist)

- Claves privadas con permisos `0600`, directorios `0700`; contenedores como usuario no root (uid 10001), con `cap_drop: ALL` y `no-new-privileges`.
- **Mínimo privilegio en el material criptográfico (Docker Compose).** Cada nodo monta un volumen propio con `ca.pem` + su certificado y clave; `cluster.key` sólo existe en los volúmenes de los 3 ControlNodes (los DataNodes nunca lo ven, coherente con §3) y `ca.key` sólo en el volumen `state` del contenedor efímero `pki-init`. Así, comprometer un DataNode no entrega ni la clave maestra ni la capacidad de emitir certificados. Una versión anterior montaba un único volumen compartido con todas las claves privadas y `ca.key`; se corrigió y quedó protegido por `tests/test_compose.py` (cada secreto tiene exactamente los titulares esperados) y por `scripts/compose_dryrun.py`.
- En AWS, `deploy/aws/state/` (CA + `cluster.key`) permanece en la máquina del operador, con permisos `0700`, y **nunca se versiona** (`.gitignore`); a cada VM sólo llega su paquete.
- Puertos internos (mTLS) cerrados a Internet (*security group* sólo desde el propio grupo).
- Rotar `cluster.key` requiere re-envolver DEK (procedimiento manual documentado en OPERACION.md; no automatizado).
- Cambiar la clave de `admin` tras el bootstrap; usar API keys de menor privilegio para scripts.

7. Limitaciones conocidas (honestas)

- El ControlNode puede desenvolver DEK (es quien las custodia): un ControlNode comprometido en ejecución ve los datos. Mitigación futura: KMS externo (p. ej. AWS KMS) para la KEK, o cifrado de extremo a extremo en el cliente.
- El bloqueo anti fuerza bruta es en memoria del líder (se reinicia con un cambio de líder).
- Revocación de certificados de nodo: no hay CRL; se resuelve reemitiendo la CA o retirando el nodo.
- Sin protección contra DoS volumétrico (se delegaría a un balanceador/WAF delante de los puertos públicos).

Entorno de ejecución: nativo y Docker

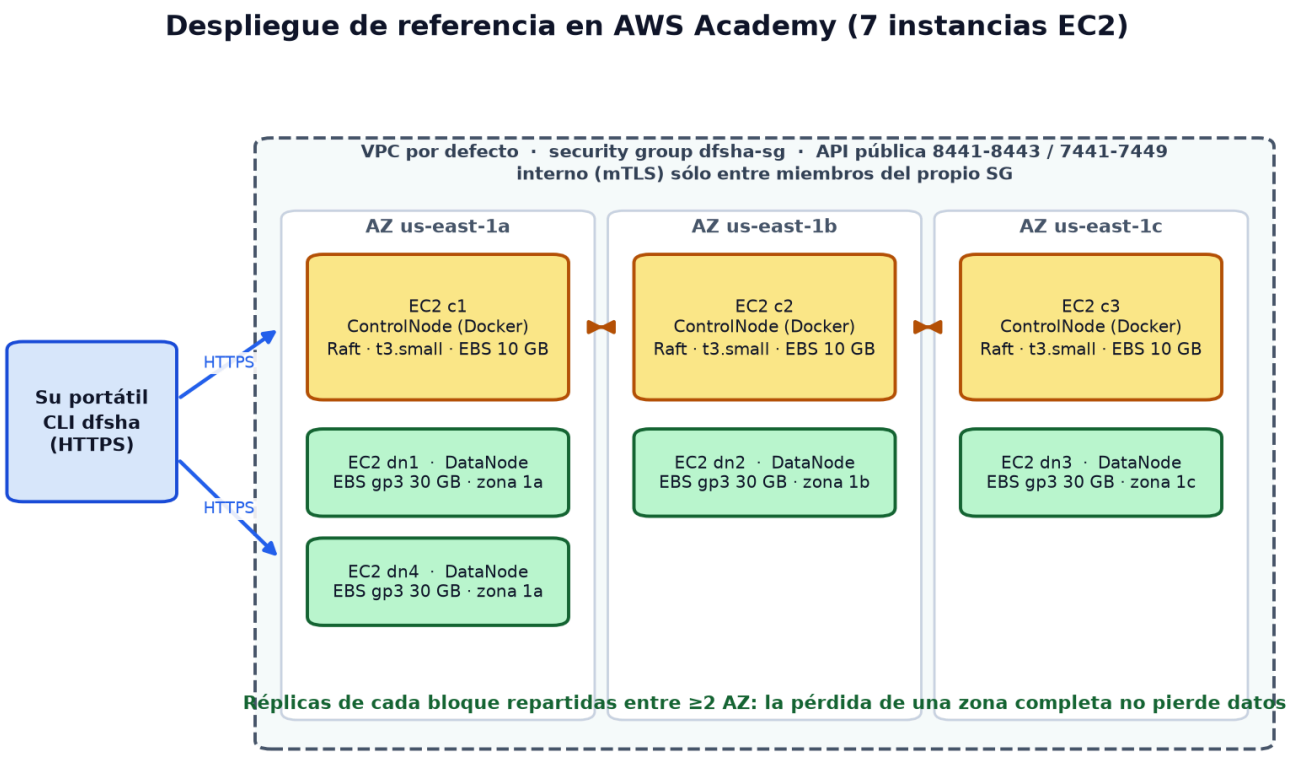


Figura 6. Despliegue de referencia en AWS Academy

Hay tres formas de ejecutar DFSha; las tres usan **exactamente el mismo código**:

Modo	Para qué	Comando
Nativo, cluster local	desarrollo, demo, tests, benchmarks	`dfsha-cluster up`
Docker Compose	red aislada, "un solo comando", entorno reproducible	`docker compose up -d --build`
IaaS (AWS Academy)	despliegue distribuido real en varias VM	`deploy/aws/`

1. Requisitos

- Python ≥ 3.12 y `uv` (`brew install uv`) — o Docker.
- Linux/macOS. El código no usa nada específico de plataforma.

```
git clone <repo> && cd SIST-DISTR
uv sync --extra dev --extra report      # crea .venv e instala todo
```

2. Modo nativo: cluster local multiproceso

```
uv run dfsha-cluster up --control 3 --data 5 --zones 2   # CA + certificados + 8 procesos
eval "$(uv run dfsha-cluster env)"                       # DFSHA_ENDPOINTS, DFSHA_CA, DFSHA_USER...
uv run dfsha login -u admin -p dfsha-admin-dev           # clave de desarrollo por defecto
uv run dfsha nodes                                       # ControlNodes (Raft) y DataNodes
uv run dfsha put ./archivo.bin /home/admin/archivo.bin
uv run dfsha blocks /home/admin/archivo.bin              # dónde está cada réplica
uv run dfsha get /home/admin/archivo.bin ./copia.bin
uv run dfsha shell                                       # shell interactivo
```

Caos y gestión del cluster local:

```
uv run dfsha-cluster status
uv run dfsha-cluster kill dn2        # SIGKILL a un DataNode
uv run dfsha-cluster kill leader     # SIGKILL al líder Raft
uv run dfsha-cluster start dn2       # lo vuelve a levantar
uv run dfsha-cluster down            # detiene (conserva datos)    |   destroy: borra todo
```

Cada nodo escribe su log en `.cluster/logs/<nodo>.log` y sus datos en `.cluster/data/<nodo>/`.

3. Docker Compose (3 ControlNodes + 5 DataNodes + cliente)

```
cp .env.example .env                 # y cambie ADMIN_PASSWORD (obligatoria; el valor del ejemplo es sólo un marcador)
docker compose build
docker compose up -d                 # pki-init genera CA/certificados/secreto; luego arrancan los nodos
docker compose ps                    # los 8 nodos deben quedar (healthy)
docker compose --profile client up -d client
docker compose exec client dfsha nodes
docker compose exec client dfsha put /work/archivo.bin /home/admin/archivo.bin
```

- Un contenedor por nodo (misma imagen `dfsha:latest`, distinto comando), red `dfsha-net`, usuario sin privilegios (uid 10001), `cap_drop: ALL`, `no-new-privileges`, puertos **internos (mTLS) sin publicar**.
- **Aislamiento de claves (mínimo privilegio).** El `docker-compose.yml` lo genera `docker/gen_compose.py` (no se edita a mano; `tests/test_compose.py` comprueba que está sincronizado). Cada nodo monta **sólo** su volumen `pki-<nodo>` (`ca.pem` + su propio `.crt/.key`); `cluster.key` y `admin.pass` están únicamente en `sec-<controlnode>` (los DataNodes jamás lo montan); `ca.key` vive sólo en el volumen `state`, privado del contenedor efímero `pki-init`. El cliente recibe un volumen `pki-client` con la CA pública. Comprobación: `docker exec dfsha-dn1 ls /pki /secrets` → `ca.pem dn1.crt dn1.key` y `/secrets` vacío.
- `state` conserva la CA y `cluster.key`: **no lo borre** si quiere conservar los datos almacenados (`docker compose down` lo respeta; `down -v` lo destruye todo, datos incluidos).
- Los nodos se anuncian por su nombre de servicio (`https://dn1:7441`): el contenedor **cliente** está en esa red y es la vía recomendada. Para usar el CLI desde el anfitrión haga falta que esos nombres resuelvan a `127.0.0.1` (línea `127.0.0.1 c1 c2 c3 dn1 dn2 dn3 dn4 dn5` en `/etc/hosts`; los puertos 8441-8443 y 7441-7445 están publicados) y copie la CA del volumen de cliente: `docker run --rm -v dfsha_pki-client:/pki alpine cat /pki/ca.pem > ca.pem`; luego `DFSHA_ENDPOINTS=https://c1:8441,https://c2:8442,https://c3:8443 DFSHA_CA=./ca.pem dfsha login`.
- Caos: `docker compose kill dn2`, `docker compose stop c1`, `docker compose start c1`.
- **Estado de verificación.** Docker no estaba instalado en la máquina de desarrollo, por lo que la imagen y los contenedores **no se construyeron ni ejecutaron**. La configuración se validó por otras dos vías: 1. `tests/test_compose.py` comprueba la estructura del compose y el reparto de claves (cada nodo monta sólo las suyas). 2. `scripts/compose_dryrun.py` **lee el propio `docker-compose.yml`**, genera la PKI con `docker/init-pki.sh` hacia directorios por servicio (mismo reparto de volúmenes), arranca los 8 nodos usando únicamente las variables de entorno, puertos y URLs del compose (con los nombres de servicio resolviendo a 127.0.0.1), comprueba que ningún servicio ve claves ajenas, verifica `put`/`get` y la caída del líder.

Quedan sin comprobar el `Dockerfile`, los permisos reales de los volúmenes y la red entre contenedores.

4. AWS Academy (IaaS)

Los scripts están en `deploy/aws/` del repositorio. Resumen: 3 EC2 ControlNode + N EC2 DataNode en distintas zonas de disponibilidad (más una VM cliente dentro de la VPC para medir sin la latencia de su casa), un *security group* que expone sólo las APIs de cliente (desde `ALLOW_CIDR`) y deja los puertos internos mTLS únicamente entre miembros del grupo; certificados con SAN de IP privada y DNS público; contenedores con `--restart unless-stopped`.

Script (`deploy/aws/`)	Función
`preflight.sh`	comprueba CLI, credenciales del Learner Lab, clave PEM, región y cuotas **antes** de gastar nada
`launch.sh` / `refresh-inventory.sh`	crea SG + instancias y escribe `inventory.txt` (se refresca si las IP cambian al reiniciar el lab)
`gen-bundles.sh`	PKI y paquetes por nodo en `state/` (la CA y `cluster.key` **no** se regeneran: sin ellas los datos serían ilegibles)
`deploy.sh`	copia y arranca cada nodo; espera a que se forme el cluster
`status.sh` / `node.sh`	estado y operaciones por nodo (`kill`, `stop`, `start`, `logs`, `blk`, `leader` …)
`validate.sh`	validación de 10 pasos con caos real (`dfsha-cloud validate`), evidencia en `deploy/aws/evidence/<fecha>/`
`bench-remote.sh`	benchmarks desde la VM cliente (`dfsha-cloud bench`), resultados en `bench/results_cloud/`
`teardown.sh`	elimina instancias y SG (hágalo al terminar para no consumir créditos)

5. Referencia de configuración

Archivos de ejemplo (`*.env.example`)

Copie el que necesite sin la palabra `example` y edítelo; las copias reales (`.env`, `.env.client`, `deploy/aws/.env`) **no se versionan** (están en `.gitignore`).

Archivo	Para qué	Cómo se usa
`.env.example`	Docker Compose (`ADMIN_PASSWORD`, replicación, tamaño de bloque, capacidad)	`cp .env.example .env` y `docker compose up -d`
`.env.client.example`	cliente (`dfsha` /SDK): endpoints, CA, usuario	`set -a; source .env.client; set +a` (PowerShell: ver el archivo)
`deploy/aws/.env.example`	scripts de AWS: región, clave SSH, IP permitida, tamaño del despliegue	`cp deploy/aws/.env.example deploy/aws/.env`; los scripts lo leen solos (lo exportado en el terminal tiene prioridad)
`docker/control.env.example`	todas las variables de un ControlNode	para arrancar `dfsha-control` a mano
`docker/data.env.example`	todas las variables de un DataNode	para arrancar `dfsha-data` a mano

ControlNode (`dfsha-control`, variables `DFSHA_*` o flags)

Opción	Defecto	Descripción
`--id`, `--peers`	–	id propio y lista `id=url_publica,url_interna;…` de **todos** los ControlNodes
`--cert/--key/--ca`	–	certificado del nodo (rol `controlnode`) y CA
`--cluster-secret`	–	archivo con el secreto maestro (idéntico en los 3 ControlNodes)

Opción	Defecto	Descripción
`--admin-password[-file]`	–	clave del usuario `admin` en el primer arranque
`--repl`	3	factor de replicación por defecto
`--block-size`	4MiB	tamaño de bloque por defecto
`--write-quorum`	majority	`majority` | `all` | `one`
`--election-timeout min max` / `--heartbeat`	1.0 2.0 / 0.15	temporización de Raft (s)
`--snapshot-threshold`	2000	entradas del log antes de compactar
`--dn-dead-after`	10	s sin heartbeat para declarar muerto un DataNode
`--repl-grace`	15	s de espera antes de re-replicar tras una caída
`--gc-grace`	120	s de gracia antes de borrar físicamente bloques huérfanos
`--balance-interval` / `--balance-tolerance`	30 / 0.2	balanceador (0 = off)
`--no-fsync`	–	**sólo pruebas**: desactiva fsync del log Raft

DataNode (`dfsha-data`)

Opción	Defecto	Descripción
`--id`, `--controls`	–	id y URLs **internas** de los ControlNodes
`--zone`	default	zona/rack (para colocación de réplicas)
`--advertise-public`, `--advertise-internal`	–	URLs con que clientes / nodos lo alcanzan
`--capacity`	0	tope de bytes (0 = espacio libre del disco)
`--heartbeat` / `--report-interval` / `--scrub-interval`	2 / 60 / 3600	s

6. Procedimientos (runbooks)

Tarea	Procedimiento
Añadir capacidad	Emitir certificado (`dfsha-pki issue --role datanode`), arrancar el DataNode con `--controls …`. Se registra solo; el balanceador migra datos hacia él
Retirar un DataNode	`dfsha dn-state dnX decommissioning` → esperar a que `dfsha fsck` quede sano → `dfsha dn-state dnX removed`
Reemplazar un ControlNode caído	Volver a levantarlo con su directorio de datos: se pone al día por log o `InstallSnapshot`. (La membresía Raft es estática: para cambiar la lista de nodos reinicie los 3 con la nueva lista)
Comprobar salud	`dfsha nodes`, `dfsha fsck`, `GET /v1/cluster`, `GET /metrics` (mTLS)
Verificar integridad en reposo de un DataNode	`dfsha scrub dnX` (los blobs corruptos pasan a cuarentena y se re-replican)
Backup de metadatos	Copiar `data/<cN>/raft/` de un seguidor (snapshot + log) con el nodo detenido
Rotar el certificado de un nodo	`dfsha-pki issue` con el mismo `--name`; reiniciar el nodo
Rotar `cluster.key`	No automatizado: requiere re-envolver las DEK de todos los archivos (limitación documentada)
Recuperación ante desastre total	Restaurar `raft/` y los datos de los DataNodes; el informe de bloques reconcilia

7. Observabilidad

- Logs por nodo (`INFO` por defecto); `audit.log` en cada ControlNode.
- `GET /metrics` (puerto interno, mTLS) — Prometheus: término Raft, líder, índice de commit, bloques, DataNodes vivos, bloques escritos/leídos/replicados por DataNode, bytes entrantes/salientes.
- `dfsha nodes` resume en una pantalla el estado de ambos planos.

Despliegue en la nube (AWS Academy)

El sistema se desplegó en máquinas virtuales EC2 de AWS Academy (Learner Lab) con los scripts de `deploy/aws/` y se validó el 2026-10-14 19:41:04 con `dfsha-cloud validate` desde un cliente externo, con tráfico real sobre Internet. Cada nodo corre en su propio contenedor Docker con la misma imagen.

Rol	Nodo	Zona de disponibilidad	Dirección pública
client	cl1	us-east-1f	`ec2-98-92-16-167.compute-1.amazonaws.com`
control	c1	us-east-1f	`ec2-98-92-33-233.compute-1.amazonaws.com`
control	c2	us-east-1d	`ec2-3-89-191-250.compute-1.amazonaws.com`
control	c3	us-east-1a	`ec2-3-234-249-169.compute-1.amazonaws.com`
data	dn1	us-east-1f	`ec2-44-223-43-56.compute-1.amazonaws.com`
data	dn2	us-east-1d	`ec2-98-81-1-135.compute-1.amazonaws.com`
data	dn3	us-east-1a	`ec2-98-92-235-71.compute-1.amazonaws.com`
data	dn4	us-east-1b	`ec2-34-226-248-193.compute-1.amazonaws.com`

Los DataNodes abarcan 4 zonas de disponibilidad (us-east-1a, us-east-1b, us-east-1d, us-east-1f); la colocación de réplicas usa cada AZ como *zona*, de modo que la pérdida de una AZ completa no destruye datos.

Validación en la nube

Paso	Resultado	Detalle	s
1. Conectividad y líder único	**PASS**	3 ControlNodes responden por HTTPS verificado contra la CA; líder = c2 (término 1)	0.93
2. Login y topología	**PASS**	4 DataNodes vivos en 4 zonas ['us-east-1a', 'us-east-1b', 'us-east-1d', 'us-east-1f']	0.19
3. Controles de seguridad de red	**PASS**	http_plano: rechazado; tls_sin_ca: rechazado (CERTIFICATE_VERIFY_FAILED); api_sin_token: HTTP 503; bloque_sin_token: HTTP 401; puerto_interno: no alcanzable desde Internet (security group correcto)	5.31
4. put/get con verificación SHA-256	**PASS**	64 MiB en 17 bloques; subida 8 MiB/s, bajada 2 MiB/s; SHA-256 idéntico	33.99
5. Mapa de bloques (réplicas y zonas)	**PASS**	17 bloques × 3 réplicas, repartidos en 4 DataNodes ['dn1', 'dn2', 'dn3', 'dn4']	0.45
6. Cifrado en reposo en el disco	**PASS**	dn1: 12 bloques en disco, cabecera DFSB (AES-GCM) y 0 coincidencias del marcador en claro que se subió	1.71
7. Failover del líder	**PASS**	líder c2 detenido → escritura aceptada otra vez en 5.6s por el nuevo líder c1; datos íntegros; c2 se reincorporó como seguidor en 3s	37.44
8. Caída de un DataNode	**PASS**	dn1 detenido → lectura inmediata correcta (30.6s); subreplicación detectada a 31s; 3 réplicas vivas por bloque restablecidas a 31s	37.43
9. fsck final	**PASS**	18 bloques, todos sanos	0.08
10. Limpieza	**PASS**	archivos de prueba eliminados	0.44

Veredicto: todos los pasos superados.

- **Transferencia sobre Internet** (64,0 MiB en 17 bloques): subida 7,8 MiB/s y bajada 2,5 MiB/s desde el cliente externo, con SHA-256 idéntico.
- **Failover del líder en la nube**: se detuvo el contenedor del líder `c2` y el servicio volvió a aceptar escrituras en **5,6 s** con el nuevo líder `c1`; los datos previos siguieron íntegros y el nodo se reincorporó como seguidor en 3,1 s.
- **Caída de un DataNode**: tras detener `dn1`, la lectura inmediata fue correcta (30,6 s); la subreplicación se detectó a los 31,0 s y se restablecieron 3 réplicas vivas por bloque a los **31,2 s** (con `dn_dead_after`=10 s y `repl_grace`=15 s, los valores por defecto de los contenedores).

Rendimiento con nodos en máquinas distintas

Medido desde una VM cliente del mismo VPC (4 DataNodes en total, 2026-10-15 00:53:05); se varía el número de DataNodes *activos* excluyendo al resto con `decommissioning` (los contenedores no se tocan).

Replicación	DataNodes activos	Subida MiB/s	Bajada MiB/s
×1	1	81,5 ± 0,7	104 ± 0,6
×1	2	114 ± 20,0	114 ± 3,9
×1	3	135 ± 7,8	119 ± 1,9
×1	4	139 ± 36,3	114 ± 4,2
×3	3	55,4 ± 0,6	117 ± 2,4
×3	4	60,3 ± 1,7	115 ± 2,2



Figura 7. Escalado en la nube frente a una sola máquina

Con replicación ×1, pasar de 1 a 4 DataNodes cambia la subida de 81,5 a 139 MiB/s (1,7×). A diferencia del experimento de una sola máquina, aquí cada DataNode tiene su propia CPU, red y disco, por lo que el escalado refleja el comportamiento distribuido real.

La **bajada** apenas cambia con el número de DataNodes (entre 104 y 119 MiB/s): a partir de ahí el límite deja de estar en el cluster y pasa a la única VM cliente (CPU para TLS y AES-GCM, o su red). No se aisló cuál de las dos; para medirlo habría que lanzar varios clientes en paralelo. Con replicación ×3 la subida es menor porque cada bloque se escribe tres veces (tubería de replicación).

Efecto del paralelismo y del tamaño de archivo

Configuración	Subida MiB/s	Bajada MiB/s
workers=8,size=256MiB	59,6	114

Operaciones de metadatos en la nube

Clientes	stat linealizable ops/s (p99 ms)	mkdir Raft ops/s (p99 ms)
1	518 (3,3)	78,4 (19,5)
8	900 (13,8)	211 (54,4)
32	526 (298)	328 (172)

Evidencia gráfica

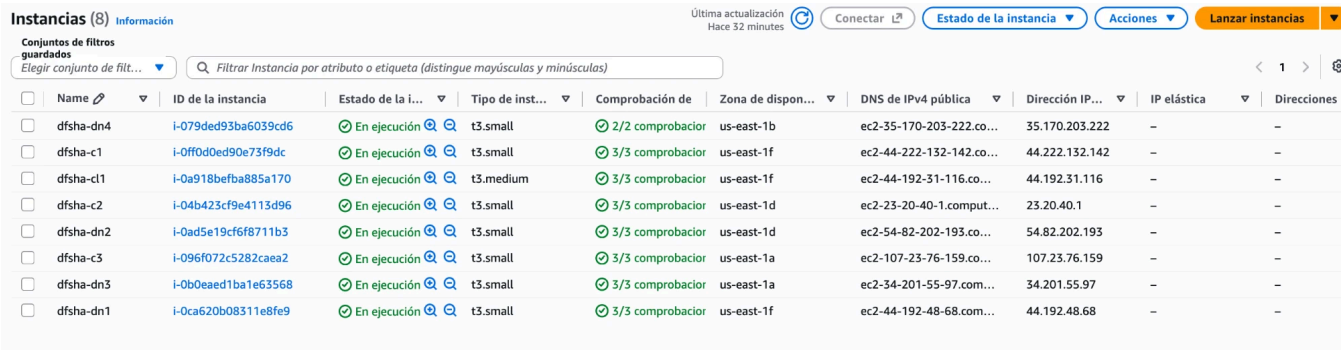


Figura 8. Consola EC2 las 8 instancias en ejecucion en distintas zonas

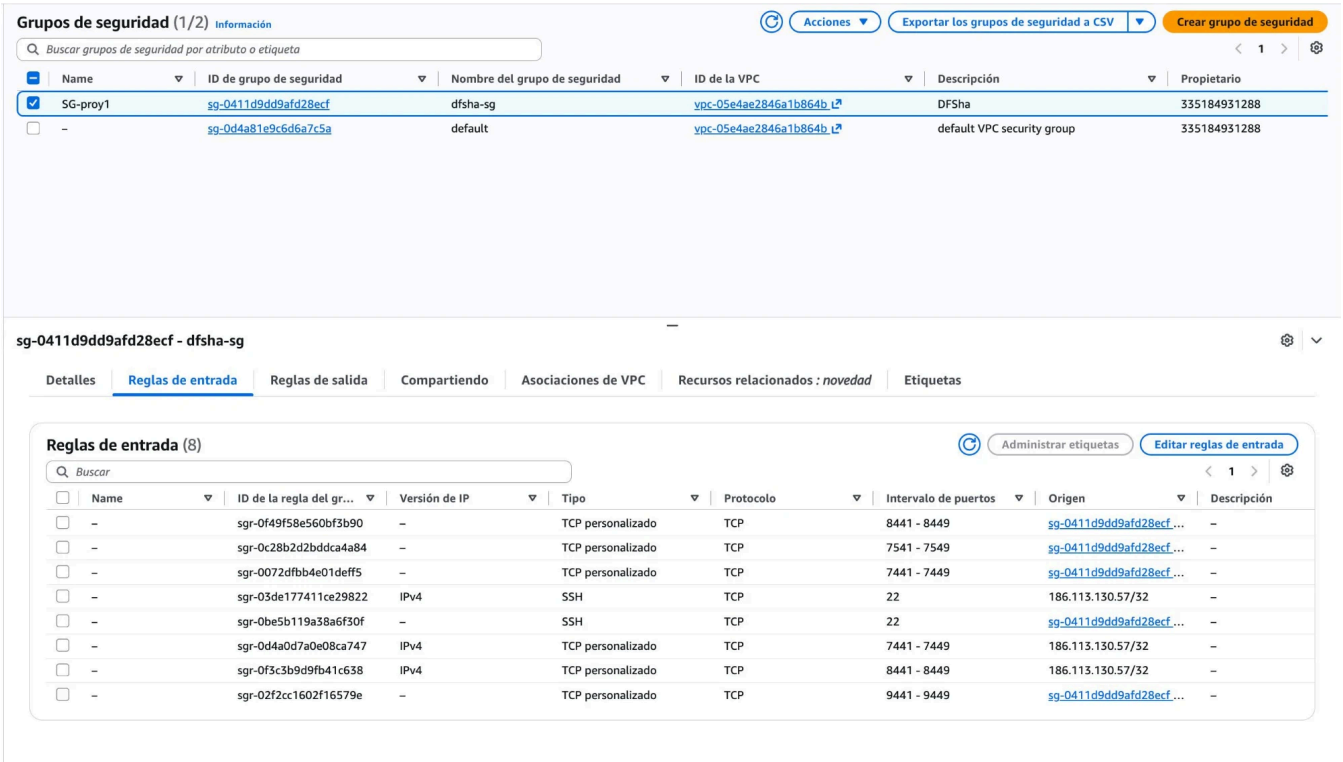


Figura 9. Security group reglas de entrada

```
uv run dfsha nodes
```

ControlNodes (Raft)

id	rol	término	commit	líder
c1	leader	2	6866	c1
c2	follower	2	6866	c1
c3	follower	2	6866	c1

DataNodes

id	zona	estado	vivo	bloques	usado	capacidad	carga
dn1	us-east-1f	active	sí	0	0 B	20.0 GiB	0
dn2	us-east-1d	active	sí	0	0 B	20.0 GiB	0
dn3	us-east-1a	active	sí	0	0 B	20.0 GiB	0
dn4	us-east-1b	active	sí	0	0 B	20.0 GiB	0

```
bloques=0 sanos=0 sub-replicados=0 perdidos=0  | archivos=0  |
mantenimiento={'replicated': 13, 'replicate_failed': 0, 'gc_deleted': 5059,
'trimmed': 13, 'balanced': 0}
```

Figura 10. dfsha nodes lider seguidores y DataNodes por zona

```
uv run dfsha blocks /home/admin/p.bin
```

/home/admin/p.bin 45.8 MiB v1 replicación=3 bloque=4.0 MiB

#	bloque	bytes	sha256	réplicas (DataNode@zona)
0	7e68d82beb	4194304	88525b4d4551	dn4@us-east-1b dn1@us-east-1f dn3@us-east-1a
1	5e9100ae64	4194304	93f2e0aaebb4	dn1@us-east-1f dn2@us-east-1d dn3@us-east-1a
2	1edfad9ea1	4194304	23d1f8245497	dn3@us-east-1a dn2@us-east-1d dn4@us-east-1b
3	6f02c9f842	4194304	43085c24f7dc	dn1@us-east-1f dn4@us-east-1b dn2@us-east-1d
4	31c6a257f5	4194304	eb0f883c7bda	dn3@us-east-1a dn2@us-east-1d dn1@us-east-1f
5	b9f55db8c4	4194304	2312c7ad7b8f	dn2@us-east-1d dn1@us-east-1f dn4@us-east-1b
6	5d64aa02d7	4194304	6574a630c34a	dn4@us-east-1b dn2@us-east-1d dn1@us-east-1f
7	fffced2cc3	4194304	35aef0afa5ca	dn3@us-east-1a dn4@us-east-1b dn1@us-east-1f
8	a3f21b777d	4194304	084956864ca5	dn2@us-east-1d dn1@us-east-1f dn3@us-east-1a
9	8d72d1c01d	4194304	0e05af25e79d	dn1@us-east-1f dn2@us-east-1d dn4@us-east-1b
10	55481bf2af	4194304	99818816f2c5	dn1@us-east-1f dn3@us-east-1a dn4@us-east-1b
11	264daf85c7	1862656	fdef9b8ddda1	dn4@us-east-1b dn3@us-east-1a dn2@us-east-1d

Figura 11. dfsha blocks replicas en distintas zonas

```
deploy/aws/node.sh blk dn1

/opt/dfsha/data/blocks/b9/b9f55db8c42545e99ad73ff00e1381c3.blk
00000000: 4446 5342 0100 0000 012f 762f 51ae a424  DFSB...../v/Q..$
00000010: 2f79 177a 830c e960 dbd1 4c0d c316 8465  /y.z...`..L....e
00000020: 5c4e 8767 8e43 5cc2 8f07 cb12 9519 44b1  \N.g.C\.......D.
00000030: 8505 5eb7 a3ca f625 63fe cbad 5f55 0374  ..^....%c..._U.t
00000040: 85f7 fba6 e3ac 3688 595e 3c8b a9dd fc26  ......6.Y^<....&
00000050: 234a 2eb4 ebe2 2084 b9ad 4d78 9db9 b623  #J.... ...Mx...#
```

Figura 12. Cifrado en reposo cabecera DFSB en el disco de un DataNode

```
~/Desktop/Gonzo/EF/SIST-DISTR git:(main) 0 • +0 -0 (2.527s)
deploy/aws/node.sh kill $(deploy/aws/node.sh leader)

dfsha-node
c1: contenedor terminado (SIGKILL)

~/Desktop/Gonzo/EF/SIST-DISTR git:(main) 0 • +0 -0 (1.905s)
uv run dfsha nodes
```

ControlNodes (Raft)

id	rol	término	commit	líder
c1	caído	-	-	-
c2	leader	3	6979	c2
c3	follower	3	6979	c2

DataNodes

id	zona	estado	vivo	bloques	usado	capacidad	carga
dn1	us-east-1f	active	sí	7	25.8 MiB	20.0 GiB	0
dn2	us-east-1d	active	sí	9	33.8 MiB	20.0 GiB	0
dn3	us-east-1a	active	sí	12	45.8 MiB	20.0 GiB	0
dn4	us-east-1b	active	sí	9	33.8 MiB	20.0 GiB	0

```
bloques=12 sanos=12 sub-replicados=0 perdidos=0  | archivos=1  |
mantenimiento={'replicated': 0, 'replicate_failed': 0, 'gc_deleted': 0, 'trimmed':
6, 'balanced': 6}
```

Figura 13. Failover del lider Raft tras matar al lider

```
~/Desktop/Gonzo/EF/SIST-DISTR git:(main) 0 • +0 -0 (2.27s)
deploy/aws/node.sh kill dn2

dfsha-node
dn2: contenedor terminado (SIGKILL)

~/Desktop/Gonzo/EF/SIST-DISTR git:(main) 0 • +0 -0 (1.167s)
uv run dfsha fsck

bloques=12 sanos=3 sub-replicados=9 sobre-replicados=0 perdidos=0

~/Desktop/Gonzo/EF/SIST-DISTR git:(main) 0 • +0 -0 (0.991s)
uv run dfsha fsck

bloques=12 sanos=12 sub-replicados=0 sobre-replicados=0 perdidos=0
```

Figura 14. Re replicacion automatica tras matar un DataNode

```
| Paso | Resultado | Detalle | s |
|---|---|---|---|
| 1. Conectividad y líder único | **PASS** | 3 ControlNodes responden por HTTPS verificado contra la CA;
líder = c2 (término 1) | 0.93 |
| 2. Login y topología | **PASS** | 4 DataNodes vivos en 4 zonas ['us-east-1a', 'us-east-1b', 'us-east-1d
', 'us-east-1f'] | 0.19 |
| 3. Controles de seguridad de red | **PASS** | http_plano: rechazado; tls_sin_ca: rechazado (CERTIFICATE
_VERIFY_FAILED); api_sin_token: HTTP 503; bloque_sin_token: HTTP 401; puerto_interno: no alcanzable desde
 Internet (security group correcto) | 5.31 |
| 4. put/get con verificación SHA-256 | **PASS** | 64 MiB en 17 bloques; subida 8 MiB/s, bajada 2 MiB/s;
SHA-256 idéntico | 33.99 |
| 5. Mapa de bloques (réplicas y zonas) | **PASS** | 17 bloques × 3 réplicas, repartidos en 4 DataNodes [
'dn1', 'dn2', 'dn3', 'dn4'] | 0.45 |
| 6. Cifrado en reposo en el disco | **PASS** | dn1: 12 bloques en disco, cabecera DFSB (AES-GCM) y 0 coi
ncidencias del marcador en claro que se subió | 1.71 |
| 7. Failover del líder | **PASS** | líder c2 detenido → escritura aceptada otra vez en 5.6s por el nuevo
 líder c1; datos íntegros; c2 se reincorporó como seguidor en 3s | 37.44 |
| 8. Caída de un DataNode | **PASS** | dn1 detenido → lectura inmediata correcta (30.6s); subreplicación
detectada a 31s; 3 réplicas vivas por bloque restablecidas a 31s | 37.43 |
| 9. fsck final | **PASS** | 18 bloques, todos sanos | 0.08 |
| 10. Limpieza | **PASS** | archivos de prueba eliminados | 0.44 |

**Veredicto:** TODOS LOS PASOS SUPERADOS
```

Figura 15. Validacion automatica en AWS 10 pasos superados

```
{
 "env": {
  "fecha": "2026-10-15 00:53:05",
  "endpoints": [
   "https://ec2-98-92-33-233.compute-1.amazonaws.com:8441",
   "https://ec2-3-89-191-250.compute-1.amazonaws.com:8442",
   "https://ec2-3-234-249-169.compute-1.amazonaws.com:8443"
  ],
  "datanodes": 4,
  "host": "ip-172-31-71-111.ec2.internal"
 },
 "result": {
  "repl1": {
   "1": {
    "upload_MiBps": {
     "mean": 81.54822475163881,
     "sd": 0.718837717964141,
     "min": 80.83870099229176,
     "max": 82.53348129690703,
     "n": 3,
     "all": [
      81.27249196571763,
      82.53348129690703,
      80.83870099229176
     ]
    },
    "download_MiBps": {
     "mean": 103.81136324937042,
     "sd": 0.5519508759510917,
     "min": 103.1258490611029,
     "max": 104.47742728351275,
     "n": 3,
     "all": [
      103.1258490611029,
      103.83081340349558,
      104.47742728351275
     ]
    }
   },
   "2": {
```


Figura 16. Benchmark de escalado medido desde una VM del VPC

```
> uv run dfsha login -u samuel -p $env:DFSHA_PASSWORD
> uv run dfsha whoami
> uv run dfsha cat /home/admin/hola-juan.txt
esión iniciada como samuel (grupos: samuel, users); expira 2026-10-15 08:01
amuel  grupos=samuel,users  admin=False  2fa=False
fsha: permiso denegado: /home/admin/hola-juan.txt (EACCES)

S C:\Users\SAMUEL\Downloads\paquete-companero\paquete-companero\dfsha> uv run dfsha cat /home/admin/hola-juan.txt
ola desde Juan
S C:\Users\SAMUEL\Downloads\paquete-companero\paquete-companero\dfsha>
```

Figura 17. Permisos ACL usuario samuel denegado y luego permitido

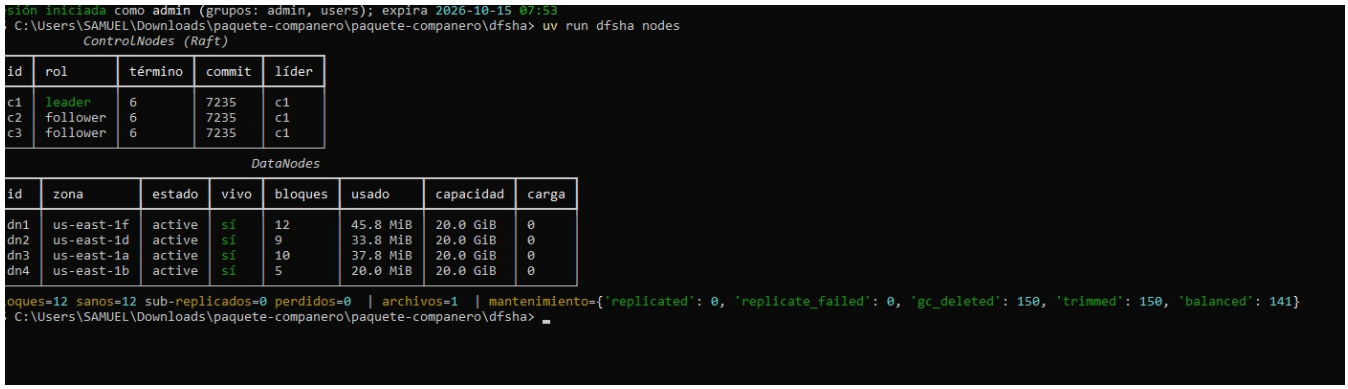


Figura 18. Acceso desde otro computador Windows al cluster en AWS

Pruebas y análisis de resultados

1. Estrategia

Pirámide de pruebas con énfasis en la corrección bajo fallos, porque ése es el riesgo principal de un sistema distribuido:

Nivel	Archivos	Qué prueba	Cómo
Unitarias de seguridad	`tests/test_security.py`	AES-GCM por bloque, AAD, bit-rot sin clave, envoltura de claves, tokens sellados (expiración, manipulación), scrypt, API keys, **TOTP contra los vectores del RFC 6238**, JWT	llamadas directas
Consenso (Raft)	`tests/test_raft.py`	elección única, replicación y orden total, caída del líder, partición minoritaria, Pre-Vote, lease de lectura, snapshots/`InstallSnapshot`, reinicio desde disco, 5 nodos tolerando 2 fallos, **caos aleatorio con invariantes**	red en memoria con particiones, caídas y reinicios controlados
Máquina de estados	`tests/test_fsm.py`	permisos POSIX/ACL, recorrido de rutas, escrituras atómicas, modos replace/append/update, CAS, sesiones y expiración, GC, locks y fencing, idempotencia, **determinismo (fuzz de 400 comandos sobre dos réplicas)**	se aplica el mismo log a réplicas independientes
Colocación	`tests/test_placement.py`	nodos distintos, diversidad de zonas, **ausencia de sesgo**, evitar nodos llenos	estadísticas sobre miles de decisiones
CLI	`tests/test_cli.py`	login, `cd` persistente, put/get/cat/cp/mv/rm, `put -r`, `blocks`, `verify`, chmod/ACL, locks, `nodes`, errores y códigos de salida	`CliRunner` contra el cluster real
Integración	`tests/test_integration.py`	cluster **real** (3 ControlNodes + 4 DataNodes, procesos separados, TLS/mTLS): idas y vueltas de 0 B a 80 MiB, distribución y zonas, **cifrado en disco**, API tipo archivo, FS, permisos y ACL, tokens, mTLS, 2FA, API keys, throttling, concurrencia, locks	`LocalCluster`
Caos / HA	`tests/test_chaos.py`	SIGKILL de DataNodes y líder **durante** subidas y bajadas, pérdida de quórum, caída total del plano de control (con fsync real), corrupción de bloques, scrubber, disco borrado, decommission, GC	un cluster nuevo por prueba
Escala	`tests/test_scale.py`	añadir un DataNode en caliente y observar el balanceo; `InstallSnapshot` real sobre HTTPS; límite de cuerpo	cluster real
Despliegue (Docker)	`tests/test_compose.py`, `tests/test_docker_verify.py`	el `docker-compose.yml` coincide con su generador; **cada nodo monta sólo sus claves** (`cluster.key` sólo en ControlNodes, `ca.key` sólo en `pki-init`); endurecimiento y puertos; `init-pki.sh` (diseño, idempotencia, cadena de certificados); la lógica del verificador de Docker	análisis del compose y un `FakeDocker`
Despliegue (AWS)	`tests/test_aws_scripts.py`, `tests/test_cloud.py`, `tests/test_demo_cloud.py`	los scripts `deploy/aws/*` con `aws`/`ssh`/`rsync` falsos (órdenes y argumentos exactos); el **validador de 10 pasos** y los benchmarks de nube ejecutados contra un cluster local que hace de «remoto»; la demo en la nube	ejecutables falsos en el `PATH` + `LocalCluster`

Principios: (1) **sin simulaciones donde se pueda usar lo real** (procesos, sockets, TLS, SIGKILL); (2) cada defecto encontrado deja una **prueba de regresión**; (3) las pruebas de caos verifican **invariantes**, no sólo "no hubo excepción": nada confirmado se pierde, ninguna entrada se aplica dos veces, todas las réplicas convergen al mismo estado, el contenido leído coincide byte a byte (SHA-256).

2. Cómo ejecutarlas

```
uv run pytest -m "not integration"       # unitarias: segundos
uv run pytest tests/test_integration.py  # ≈ 1,5 min
uv run pytest tests/test_chaos.py        # ≈ 2 min
uv run pytest tests/test_scale.py        # ≈ 1 min
uv run pytest                            # todo (≈ 4 min)
```

Verificaciones adicionales (no son parte de `pytest`)

Verificación	Cómo	Resultado
Configuración del `docker-compose.yml`	`scripts/compose_dryrun.py` (ver `OPERACION.md`)	**OK**: cluster formado sólo con las variables del compose, **aislamiento de claves por servicio**, put/get, caída del líder, healthcheck
Docker real (`docker compose up`)	—	**No ejecutado**: Docker no estaba disponible en el entorno de desarrollo
Despliegue en AWS Academy	`dfsha-cloud validate` (10 pasos con fallos reales) y benchmarks desde una VM del mismo VPC	**Ejecutado**: 10/10 pasos superados (ver capítulo «Despliegue en la nube»)
Análisis estático	`ruff check`	sin avisos

3. Defectos hallados por las pruebas (y corregidos)

#	Defecto	Cómo se detectó	Corrección	Regresión
1	La colocación («el menos lleno de dos») **nunca** elegía al nodo marginalmente más lleno: un DataNode quedaba sin datos	`test_blocks_are_partitioned…`	bandas de utilización del 5 % con desempate aleatorio	`test_placement.py`
2	`with open(p,'wb')` **confirmaba** el archivo parcial si el bloque `with` terminaba en excepción (`BufferedWriter.__exit__` no llegaba al `abort`)	`test_failed_writer_aborts…`	`_SafeBufferedWriter`/`_SafeText` abortan ante excepción; el escritor descarta datos tras `abort`	mismo test

#	Defecto	Cómo se detectó	Corrección	Regresión
3	Un seguidor con estado atrasado unos ms **rechazaba una API key recién creada** (falso `EAUTH`)	`test_api_keys` (intermitente)	los accesos no relajados se redirigen al líder **antes** de autenticar; las lecturas `stale` que fallan se reconfirman con el líder	mismo test
4	El balanceo/recorte de sobre-replicación podía **romper la diversidad de zonas**	análisis tras el test de balanceo (`zonas=2` con una zona de un solo nodo)	recorte y balanceo consultan el nº de zonas resultante	`test_scale.py`
5	Un cliente podía **declarar réplicas** en DataNodes que no se le asignaron	revisión de seguridad	`commit_write` exige `reps ⊆ asignados`	`test_fsm.py`
6	Las ubicaciones y tokens de bloque podían servirse desde estado atrasado (revocaciones tardías)	revisión de seguridad	`locations` siempre linealizable	—
7	El DataNode leía el cuerpo completo antes de comprobar el tamaño máximo del token	revisión de seguridad	`precheck_write` valida token y `Content-Length` antes de leer	`test_datanode_rejects_oversized_put…`
8	En macOS `os.fsync` **no vacía la caché del disco**: la durabilidad declarada no era real	medición (0,05 ms vs 4,3 ms)	`fsync_fd` usa `F_FULLFSYNC` en Darwin	micro-medición
9	El apagado de los nodos tardaba hasta 10 s por conexiones *keep-alive*	observando los benchmarks	`timeout_graceful_shutdown=3` y apagado paralelo	—
10	El desempate de colocación por **carga instantánea** era estricto: un DataNode con una transferencia más en el último *heartbeat* quedaba excluido de todo un lote (fallo intermitente en ≈1 de 3 ejecuciones)	repitiendo 8 veces las pruebas de integración	la carga se discretiza en escalones de 8 transferencias (empate ⇒ azar)	`test_small_load_differences_do_not_s`
11	La re-replicación tras la caída de un DataNode **se saltaba la gracia** `repl_grace` hasta ≈1 s: `dead_since()` leía la marca que anota el bucle de vivacidad (que corre cada 1 s) y, mientras no estaba anotada, caía al instante de arranque del ControlNode: la espera se daba por cumplida y se re-replicaba de inmediato por un fallo que podía ser transitorio	el validador de nube falló de forma intermitente en el paso 8 (`repaired_s` demasiado corto); se aisló con trazas de la reparación	`dead_since = last_seen + dn_dead_after` (derivado, sin depender del bucle) y registros del monitor (reparación, espera de gracia, sin destinos)	`test_repair_waits_for_the_grace_peri` (falla 2 de 6 veces con el código antiguo)
12	`docker-compose.yml` montaba **un único volumen** con `ca.key`, `cluster.key` y las claves privadas de todos los nodos en cada contenedor (un DataNode comprometido lo veía todo)	revisión de seguridad del propio compose	volúmenes por nodo generados por `docker/gen_compose.py`; `ca.key` sólo en `pki-init`	`tests/test_compose.py`
13	`test_adding_datanode_triggers_rebalancing` era intermitente: comprobaba el reparto antes de que el GC físico (`gc_grace` 120 s) borrara las réplicas sobrantes	ejecución repetida	el test usa `--gc-grace 1` y espera la convergencia con `eventually`	mismo test (3/3)
14	Al leer un archivo **sin permiso** (o inexistente) el cliente mostraba además «Exception ignored … no attribute '_ex'»: el lector de archivos fallaba en el constructor y luego no podía cerrarse	prueba de permisos en un segundo computador (Windows)	los atributos del lector se inicializan antes de consultar al ControlNode y `close()` tolera su ausencia	`test_reader_that_failed_to_open_can_`

4. Resultados

Los resultados de la última ejecución completa se registran en la sección siguiente («Benchmarks») y en la salida de `pytest`. Resumen:

Conjunto	Pruebas	Resultado
Seguridad (`test_security.py`)	13	todas pasan
Raft (`test_raft.py`)	12	todas pasan (el caos aleatorio se repitió 5 veces en ejecuciones independientes sin fallos)
Máquina de estados (`test_fsm.py`)	25	todas pasan
Colocación (`test_placement.py`)	5	todas pasan
Integración (`test_integration.py`)	39	todas pasan
CLI (`test_cli.py`)	7	todas pasan
Caos / HA (`test_chaos.py`)	16	todas pasan
Escala (`test_scale.py`)	3	todas pasan
Docker: compose y verificador (`test_compose.py`, `test_docker_verify.py`)	17	todas pasan
AWS: scripts, validador y demo en la nube (`test_aws_scripts.py`, `test_cloud.py`, `test_demo_cloud.py`)	26	todas pasan
Utilidades del repositorio (`test_report_cloud.py`, `test_check_entrega.py`, `test_env_examples.py`)	13	todas pasan
Total	**175**	**175 pasan en ≈6 min** (macOS arm64, Python 3.12, 10 CPU)

Cada prueba de integración y de caos levanta procesos reales (hasta 3 ControlNodes + 4-5 DataNodes) con TLS y mTLS, sin simulaciones. El análisis estático (`ruff`) no reporta avisos.

Entorno y metodología

- Máquina: arm64 · 10 CPU · macOS-26.6.2-arm64-arm-64bit · Python 3.12.13. **Los ControlNodes, DataNodes y clientes comparten esta única máquina** (loopback, un solo disco).

- Cada nodo es un proceso independiente con TLS/mTLS reales. Datos aleatorios incompresibles. Cada medición se repite 3 veces (media ± desviación estándar; 2 veces en los experimentos con tope). `fsync` desactivado salvo donde se indica.
- Reproducible: `uv run python bench/bench.py all` (resultados crudos en `bench/results/*.json`).
- **Salvedad metodológica**: en una sola máquina el cuello de botella es la CPU compartida (TLS, SHA-256, AES-GCM, Python), no la red ni los discos. Por eso se incluye un experimento con un **tope de ancho de banda por DataNode** (40 MiB/s, como el de una VM con red/disco propios) que sí permite observar el escalado horizontal. Las cifras absolutas deben leerse como un orden de magnitud, no como rendimiento de producción.

Costo de la criptografía por bloque (microbenchmark, 1 hilo, bloque de 4 MiB)

Operación	MiB/s	ms por bloque de 4 MiB
AES-256-GCM cifrar	2 136	1,9
AES-256-GCM descifrar	2 217	1,8
SHA-256	3 200	1,3

El cifrado en reposo y la verificación de integridad cuestan ≈3,1 ms por bloque de 4 MiB (hardware AES del procesador): **no son el cuello de botella**; el costo dominante es TLS + HTTP + copias de memoria en Python.

Memoria de los metadatos y velocidad de la máquina de estados

Concepto	Valor medido
RAM por archivo (inodo + permisos + DEK envuelta)	≈0,81 KiB
RAM por bloque (id, longitud, SHA-256, 3 réplicas)	≈0,46 KiB
Comandos aplicados por la máquina de estados (un hilo, sin red)	≈27 764 /s

Con bloques de 4 MiB el metadato ocupa ≈0,011 % del dato almacenado: **1 PiB** de datos son ≈118 GiB de RAM en el ControlNode; con bloques de 64 MiB (como HDFS) bajan a ≈7 GiB. Es el límite de escala del plano de control (un único grupo Raft con metadatos en memoria) y la razón de que el tamaño de bloque sea configurable por archivo. La máquina de estados no es el cuello de botella de las operaciones de metadatos: aplica ≈27 764 comandos/s frente a los ≈1 954 /s que se obtienen de extremo a extremo (HTTPS + Raft).

Escalado del ancho de banda con el número de DataNodes

a) Sin tope: la CPU compartida es el límite (archivo de 256 MiB, 8 transferencias paralelas)

Replicación	DataNodes	Subida MiB/s	Bajada MiB/s
×1	1	703 ± 20	718 ± 78
×1	2	780 ± 110	724 ± 25
×1	3	1087 ± 68	715 ± 13
×1	4	1029 ± 107	586 ± 61
×1	6	943 ± 79	598 ± 32
×3	3	342 ± 16	688 ± 42
×3	4	353 ± 32	648 ± 39
×3	6	388 ± 8	617 ± 71

Con un único DataNode y replicación ×1 ya se alcanzan ≈703 MiB/s: el nodo no es el límite, lo es la máquina. Con ×3 la subida baja a ≈342 MiB/s porque cada byte se escribe y se transmite tres veces por la misma CPU y el mismo disco; la bajada (≈688 MiB/s) apenas cambia porque se lee una sola réplica.

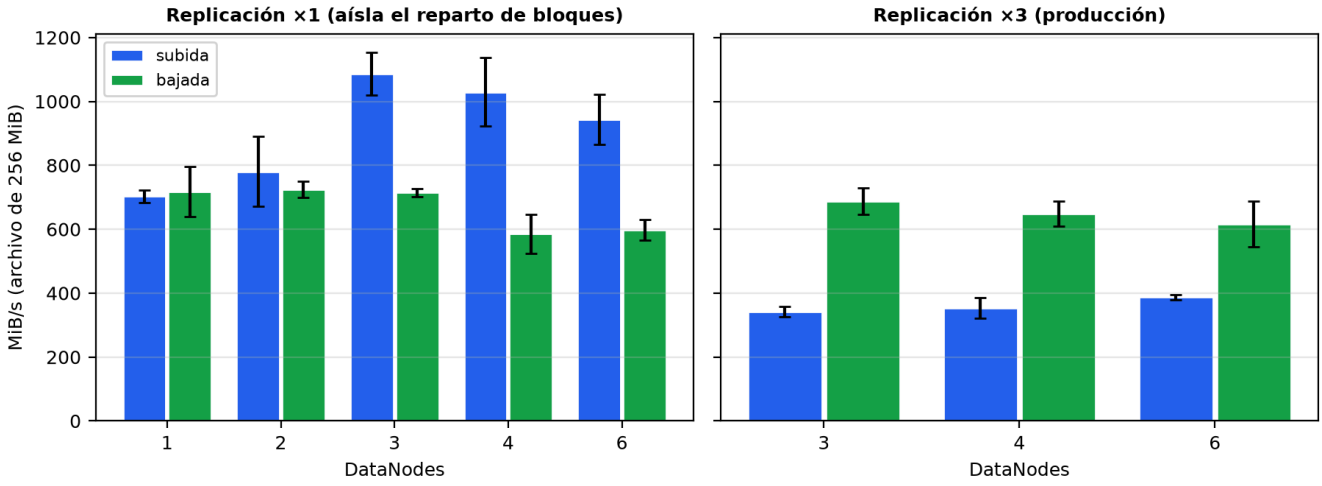


Figura 19. Throughput sin tope por nodo

b) Con tope de 40 MiB/s por DataNode (emula red/disco por nodo): escalado casi lineal

Replicación	DataNodes	Subida MiB/s	Techo teórico	Eficiencia	Bajada MiB/s
×1	1	40	40	101 %	40
×1	2	57	80	72 %	56
×1	3	87	120	73 %	84
×1	4	100	160	63 %	96
×1	6	153	240	64 %	144
×1	8	169	320	53 %	159
×3	3	38	40	95 %	105
×3	4	43	53	81 %	109
×3	6	55	80	69 %	178
×3	8	75	107	70 %	211

Con ×1, pasar de 1 a 8 DataNodes multiplica la subida por **4,2** (40 → 169 MiB/s) y la bajada de 40 a 159 MiB/s: al repartir los bloques entre más nodos, el ancho de banda agregado crece. Con ×3 el techo teórico de subida es `tope·N/3` porque cada bloque ocupa tres nodos; la eficiencia (≈70-95 %) la reduce el desbalance estadístico de una colocación aleatoria con pocos bloques por nodo y la sobrecarga fija de TLS/sesión. La bajada con ×3 escala igualmente (de 105 a 211 MiB/s), pues las lecturas se reparten entre réplicas.

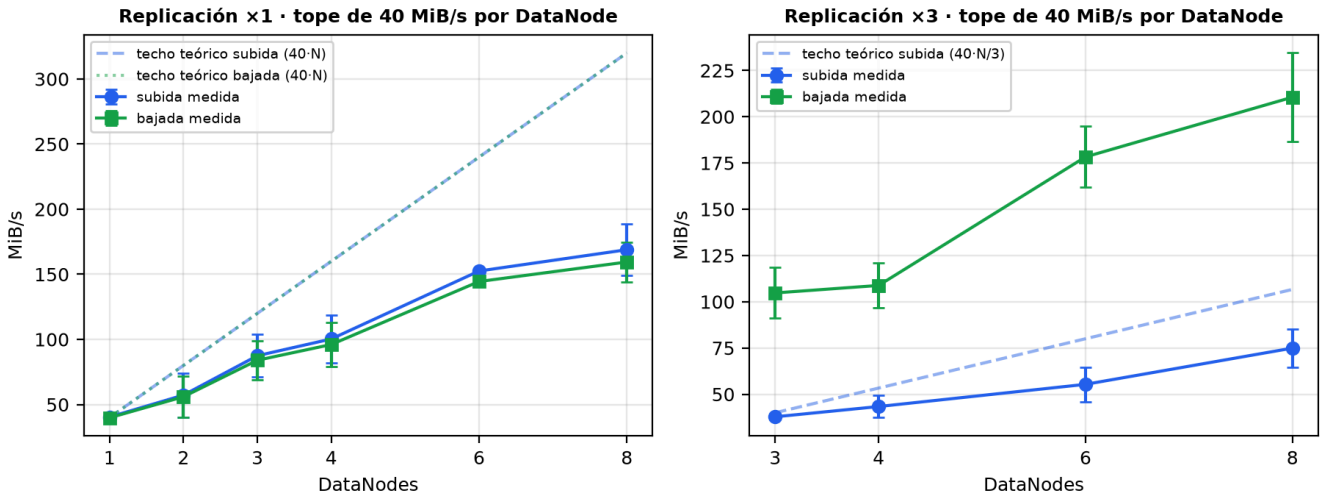


Figura 20. Throughput con tope de 40 MiB/s por DataNode

Tamaño de bloque (256 MiB, 4 DataNodes, ×3, 8 hilos)

Bloque	Bloques por archivo	Subida MiB/s	Bajada MiB/s
1MiB	256	266 ± 16	567 ± 12
2MiB	128	336 ± 20	642 ± 18
4MiB	64	366 ± 17	683 ± 25
8MiB	32	400 ± 31	654 ± 32
16MiB	16	323 ± 18	567 ± 69
32MiB	8	198 ± 49	292 ± 61

Bloques de 1 MiB pagan demasiada sobrecarga por petición; bloques de 32 MiB dejan sólo 8 bloques para 8 hilos (sin solape) y exigen mucha memoria por transferencia. La subida es máxima con 8MiB; **4 MiB** (valor por defecto) logra la mejor bajada y una subida a ≈91 % del máximo, con metadatos moderados (≈0,5 KiB de RAM por bloque de 4 MiB, ver sección anterior).

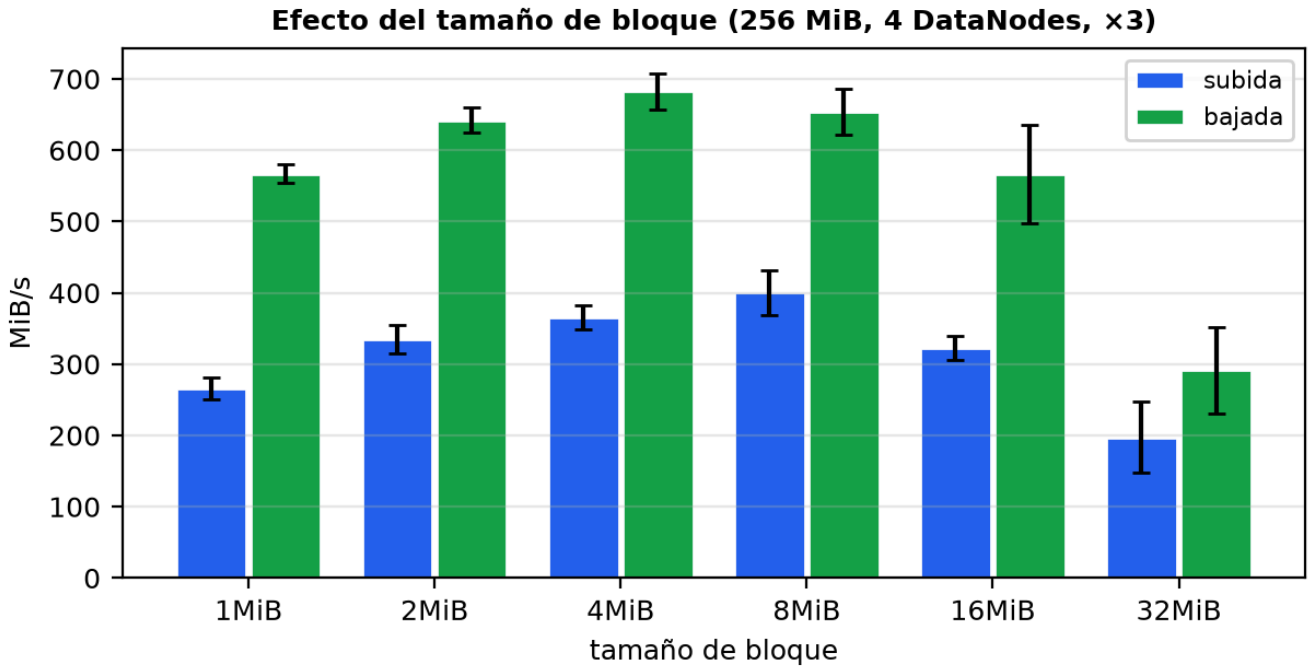


Figura 21. Tamaño de bloque

Paralelismo por bloque (256 MiB, 4 DataNodes, ×3)

Hilos	Subida MiB/s	Bajada MiB/s
1	141 ± 4	394 ± 16
2	229 ± 5	679 ± 21
4	322 ± 19	789 ± 16
8	380 ± 29	730 ± 8
16	420 ± 20	718 ± 7
32	299 ± 13	589 ± 12

Pasar de 1 a 8 hilos acelera la subida **2,7×** y la bajada **1,9×**; más allá de 16 la contención de CPU hace que el rendimiento caiga (32 hilos). El valor por defecto es 8.

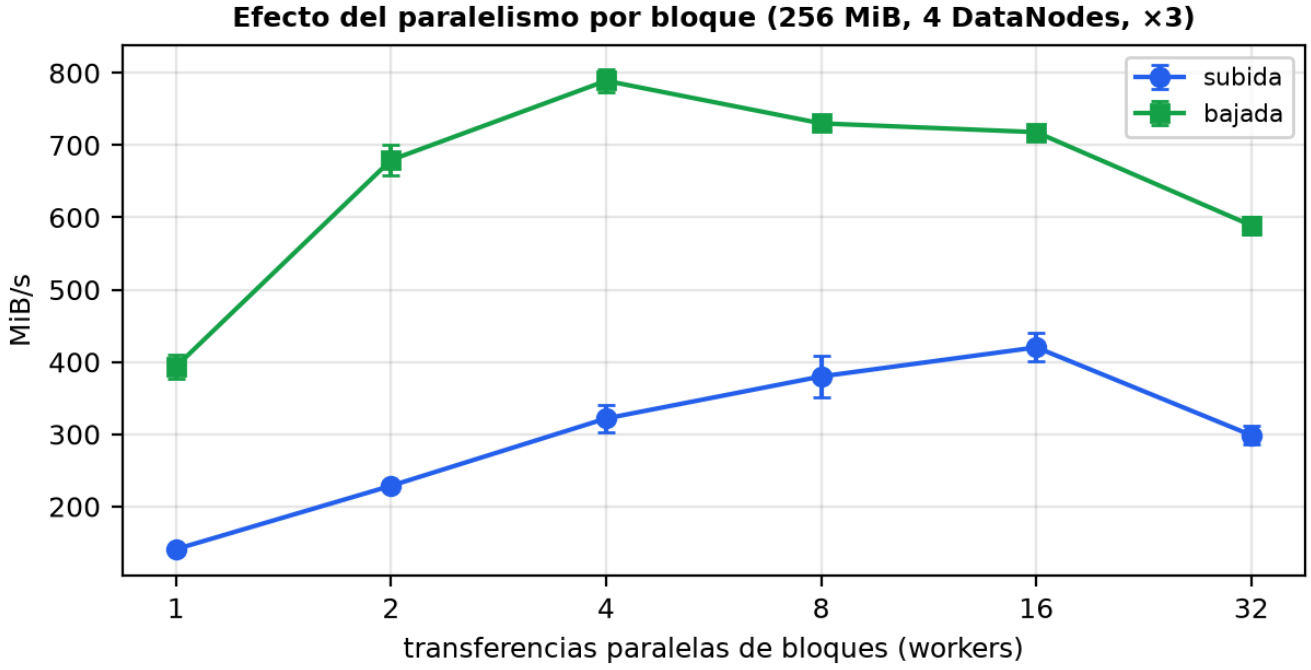


Figura 22. Paralelismo

Clientes concurrentes (6 DataNodes, ×3; cada cliente sube y baja 64 MiB)

Clientes	Throughput agregado MiB/s	Por cliente MiB/s	Tiempo total s
1	357	357	0,4
2	413	207	0,6
4	464	116	1,1
8	542	68	1,9
16	483	30	4,2

El throughput agregado crece de 357 a 542 MiB/s al pasar de 1 a 8 clientes y se estabiliza (≈483 MiB/s con 16): se satura la CPU de la única máquina que ejecuta clientes *y* servidores; en un despliegue real los clientes aportan su propia CPU y red. Todas las operaciones fueron correctas (contenido verificado byte a byte).

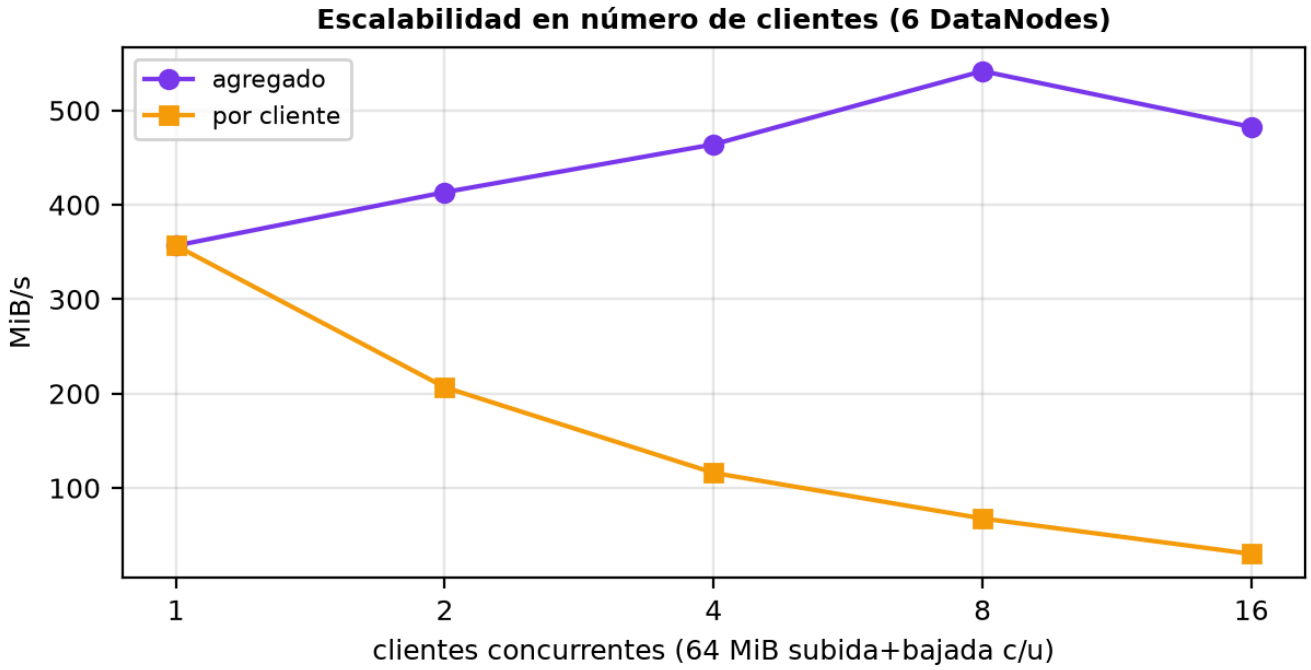


Figura 23. Clientes concurrentes

Operaciones de metadatos (3 ControlNodes)

fsync	Clientes	stat linealizable ops/s (p99)	stat relajado ops/s	mkdir (escritura Raft) ops/s (p99)
off	1	1861 (1,2 ms)	1412	412 (4 ms)
off	8	3579 (3,5 ms)	3974	1101 (22 ms)
off	32	3524 (24,7 ms)	3959	1954 (47 ms)
on	1	1772 (1,2 ms)	1597	71 (18 ms)
on	8	3025 (5,0 ms)	3692	189 (69 ms)
on	32	3260 (24,9 ms)	3996	604 (84 ms)

- Las **lecturas linealizables** las sirve el líder con *lease* (sin RPC): ≈3579 ops/s con latencia p99 de pocos ms.
- Con `fsync` **durable** (en macOS, `F_FULLFSYNC`, ≈4 ms por llamada) una escritura aislada tarda ≈14 ms (71 ops/s), pero el **group commit** agrupa las propuestas concurrentes en un único `fsync` y una única ronda de replicación: con 32 clientes se alcanzan **604 ops/s** (8,5× más). Sin fsync, 1954 ops/s.
- Las lecturas relajadas (cualquier ControlNode) no mejoran mucho en esta máquina porque el cliente y los tres ControlNodes compiten por la misma CPU; el diseño permite repartir la carga de lectura entre las réplicas.

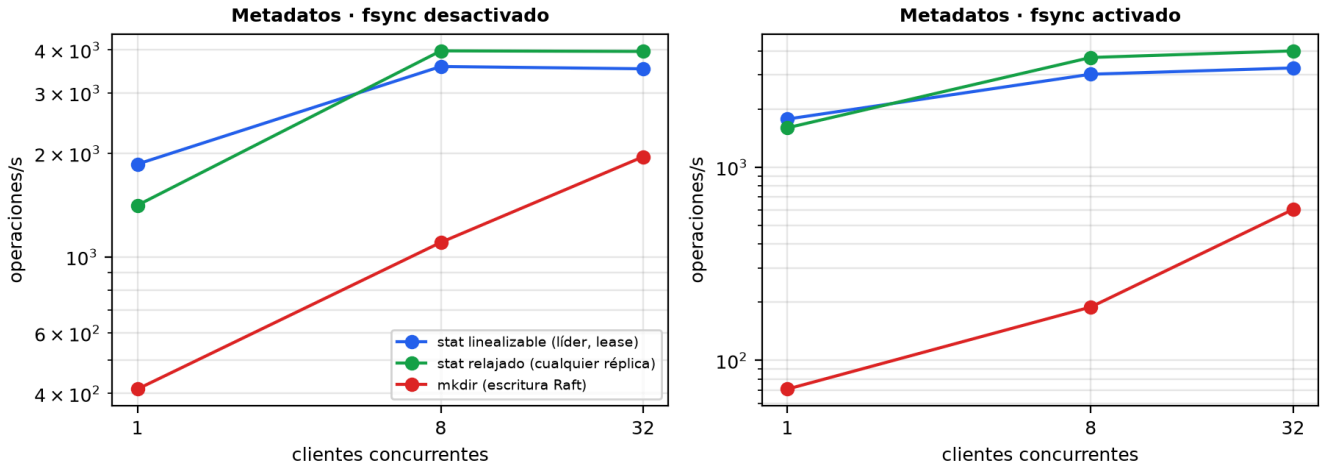


Figura 24. Operaciones de metadatos

Disponibilidad y recuperación ante fallos

- **Conmutación de líder** (SIGKILL al líder, 5 repeticiones; tiempo hasta que una escritura vuelve a tener éxito): 1,53, 3,14, 3,94, 1,54, 1,54 s → mediana **1,54 s**, máximo 3,94 s (temporizadores de elección de 0,5–1,0 s en el cluster de pruebas).
- **Re-replicación** (256 MiB, SIGKILL a un DataNode): el cluster detecta la subreplicación a los 5,4 s y recupera las 3 réplicas vivas de todos los bloques a los **9,8 s** (incluye `dn_dead_after`=6 s y `repl_grace`=3 s).
- **Carga continua durante fallos** (72 s, 3 clientes leyendo y 3 escribiendo 2 MiB sin pausa) con este plan: 8s kill dn1; 16s start dn1; 24s kill leader c2; 34s start c2; 40s kill dn2; 48s kill follower c1; 58s start all. Resultado: **14580 lecturas y 4137 escrituras, 0 fallos → disponibilidad 100,00 % / 100,00 %**. Latencia de lectura p50 14 ms, p99 28 ms, máx. 2,4 s; escritura p50 47 ms, p99 99 ms, máx. 2,4 s. Los máximos corresponden a las operaciones en vuelo cuando cae el líder (reintentos transparentes del cliente).

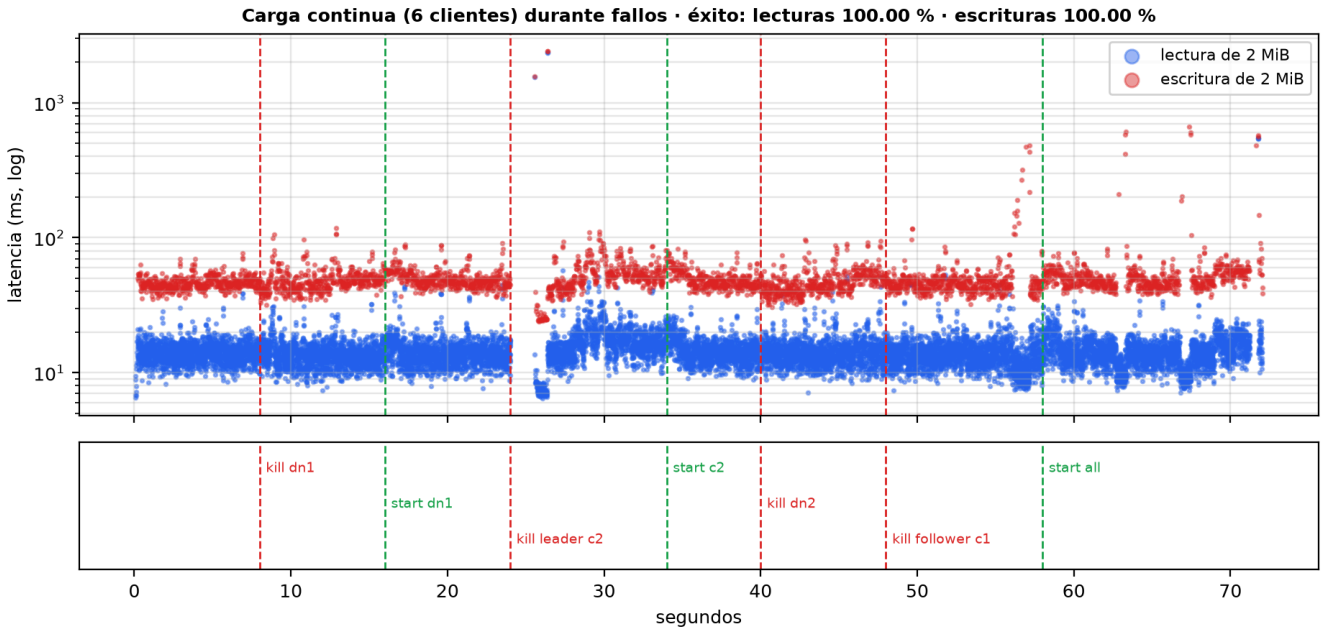


Figura 25. Disponibilidad durante fallos

Conclusiones de los benchmarks

1. El **diseño escala horizontalmente**: al repartir bloques entre más DataNodes el ancho de banda agregado crece casi linealmente cuando cada nodo tiene capacidad propia (experimento con tope), tanto en subida como en bajada.
2. La **criptografía no es un cuello de botella** (≈3 ms por bloque de 4 MiB en un solo hilo).
3. El **paralelismo por bloque** (8 hilos) y la **tubería de replicación** son las palancas de rendimiento; el *group commit* lo es para los metadatos.
4. El **servicio no se interrumpe** ante la caída de DataNodes, del líder o de seguidores (uno a la vez, f=1): los reintentos transparentes del cliente convierten los fallos en picos de latencia, no en errores.
5. Límite de la medición: una sola máquina; las cifras absolutas son un límite inferior del comportamiento distribuido real.

Cumplimiento de los criterios de evaluación

Criterio (peso)	Evidencia en este proyecto
1. Definición del servicio y diseño arquitectónico (15 %)	Requisitos funcionales RF1–RF5 y no funcionales RNF1–RNF8 con mecanismo y verificación (cap. «Descripción del servicio»); arquitectura Cliente/Servidor con servidor compuesto y distribuido, planos de control y datos, flujos de escritura/lectura/failover/reparación (cap. «Arquitectura»)
2. Especificaciones de comunicaciones (15 %)	Tabla de enlaces (Cliente↔Control, Cliente↔Data, Control↔Control, Control↔Data, Data↔Data) con protocolo, autenticación y contenido; cada endpoint REST y mensaje Raft; formato de errores, idempotencia y tokens; justificación de REST frente a gRPC/MOM (cap. «Especificación de protocolos y APIs»)
3. Diseño para escalabilidad (15 %)	Particionamiento en bloques, colocación con diversidad de zonas y balance, E/S paralela, tubería, asignación por lotes, balanceador al añadir nodos, lecturas relajadas; evidencia: benchmarks de escalado (con y sin tope por nodo), tamaño de bloque, paralelismo, clientes y metadatos; prueba de integración del balanceo
4. Alta disponibilidad (15 %)	Raft ×3 sin punto único de fallo; replicación ×3; re-replicación y reparación; failover transparente; evidencia: 175 pruebas de caos (SIGKILL de DataNodes y líder a mitad de transferencias, pérdida de quórum, caída total del plano de control, corrupción, disco borrado) y **100 % de disponibilidad medida** (14 580 lecturas y 4 137 escrituras sin fallos) durante un plan de fallos
5. Replicación y consistencia de datos (15 %)	Metadatos linealizables (Raft con lease/check-quorum/Pre-Vote), bloques inmutables con SHA-256 extremo a extremo, commit atómico, CAS, locks con fencing, idempotencia; evidencia: 175 pruebas de Raft con caos aleatorio (invariante: ningún commit confirmado se pierde), prueba de determinismo de la máquina de estados, pruebas de integración de concurrencia
6. Seguridad (15 %)	TLS en todos los enlaces, mTLS con roles, cifrado en reposo con sobre de claves, 2FA, API keys, scrypt, ACL, tokens sellados, throttling, auditoría; evidencia: pruebas que verifican cifrado en disco, rechazo de tokens ajenos/falsos, mTLS con certificados incorrectos, ACL entre usuarios, fuerza bruta y 2FA
7. Video e informe final (10 %)	Este informe y el video de demostración; repositorio documentado y reproducible (`README.md`, `uv sync`, `dfsha-cluster up`)

Limitaciones y trabajo futuro

Limitación	Impacto	Mejora propuesta
Membresía de Raft estática	Cambiar la lista de ControlNodes exige reiniciar el grupo	Cambios de configuración de un servidor a la vez (tesis de Raft §4) con nodos *learner*
Un solo grupo Raft	Techo de metadatos por la RAM y el throughput de un líder (≈0,6 k escrituras/s con fsync durable y ≈2 k sin fsync, medidas con 32 clientes)	Particionar el namespace en varios grupos Raft (federación)
Cifrado en reposo con claves custodiadas por el ControlNode	Un ControlNode comprometido en ejecución ve los datos	KEK en un KMS externo (AWS KMS / Vault) o cifrado de extremo a extremo en el cliente
Sin integración con el SO (FUSE/NFS)	Transparencia de acceso por SDK/CLI/shell, no a nivel de sistema operativo	Puerta de enlace WebDAV/NFS o montaje FUSE sobre el SDK
Sin cuotas por usuario ni límites de tasa	Un usuario puede consumir toda la capacidad	Cuotas de bytes/archivos en la máquina de estados
Replicación simple ×R	3× de almacenamiento	*Erasure coding* (p. ej. RS 6+3) para datos fríos
Rotación de `cluster.key` manual	—	Reenvolver las DEK en línea con versionado de KEK

Estado de verificación de los artefactos de despliegue

Componente	Estado	Evidencia
Cluster local multiproceso (TLS/mTLS reales)	**Verificado.** Suite automática de pruebas unitarias, de integración, de caos y de escala; benchmarks reproducibles.	`tests/`, `bench/`
Configuración del `docker-compose.yml`	**Verificada sin contenedores**: `scripts/compose_dryrun.py` arranca los 8 nodos con exactamente las variables, URLs, volúmenes por nodo y puertos del compose, y comprueba el aislamiento de claves; `tests/test_compose.py` lo verifica estructuralmente.	`scripts/compose_dryrun.py`
Imagen Docker y contenedores	**Pendiente.** No se ejecutó: Docker no estaba disponible en el entorno de desarrollo.	`docs/evidence/docker/`
Scripts de AWS (`deploy/aws/`)	**Probados con comandos simulados** (`aws`, `ssh`, `rsync`, `curl` falsos): inventario y reparto por zonas, certificados con SAN reales, mínimo privilegio por rol, persistencia de la CA entre redespliegues.	`tests/test_aws_scripts.py`
Despliegue real en AWS Academy / GCP	**Verificado el 2026-10-14 19:41:04**: topología, validación de 10 pasos con caos y benchmarks en máquinas distintas (ver el capítulo «Despliegue en la nube»).	`deploy/aws/evidence/`

Conclusiones

- Se diseñó e implementó DFSha, un DFS por bloques con un plano de control **sin punto único de fallo** (Raft propio, validado con pruebas de caos) y un plano de datos con **replicación en tubería**, reparación automática y **cifrado en reposo con sobres de claves**.
- La **separación de planos** y la **inmutabilidad de bloques** simplifican radicalmente la consistencia: los metadatos son linealizables y los datos se validan con un hash extremo a extremo; las operaciones complejas (reemplazo, anexado, actualización parcial) se reducen a "bloques nuevos + commit atómico".
- La **seguridad** se diseñó por capas (TLS/mTLS con roles, tokens de capacidad, AES-GCM, 2FA, ACL) y cada control tiene una prueba automática que intenta romperlo.
- Las mediciones muestran que el sistema **mantiene el servicio** (100 % de operaciones exitosas durante un plan de fallos que mata DataNodes, el líder y un seguidor), conmuta de líder con una mediana de 1,5 s y re-replica 256 MiB en ≈10 s. Todas las cifras de rendimiento se midieron en **una sola máquina**, por lo que son un límite inferior del comportamiento distribuido real y deben leerse con esa salvedad.
- Lección de ingeniería: las pruebas de integración y caos encontraron defectos que el análisis no había anticipado (sesgo de la colocación, `with open(...)` que confirmaba una escritura fallida, rechazos falsos en seguidores atrasados, `fsync` no durable en macOS); todos quedaron corregidos y con prueba de regresión.

Referencias

1. S. Ghemawat, H. Gobioff, S.-T. Leung. *The Google File System*. SOSP 2003.
2. K. Shvachko, H. Kuang, S. Radia, R. Chansler. *The Hadoop Distributed File System*. IEEE MSST 2010.
3. D. Ongaro, J. Ousterhout. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC 2014.
4. D. Ongaro. *Consensus: Bridging Theory and Practice* (tesis doctoral, Stanford, 2014) — Pre-Vote, lecturas con lease, cambios de membresía.
5. R. Sandberg et al. *Design and Implementation of the Sun Network Filesystem*. 1985.
6. J. Howard et al. *Scale and Performance in a Distributed File System* (AFS). ACM TOCS 1988.
7. M. Herlihy, J. Wing. *Linearizability: A Correctness Condition for Concurrent Objects*. ACM TOPLAS 1990.
8. S. Gilbert, N. Lynch. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. 2002.
9. M. Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing*. IEEE TPDS 2001.
10. E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, 2018.
11. M. Dworkin. *Recommendation for Block Cipher Modes of Operation: GCM and GMAC*. NIST SP 800-38D, 2007.
12. D. M'Raihi et al. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, 2011.
13. C. Percival, S. Josefsson. *The scrypt Password-Based Key Derivation Function*. RFC 7914, 2016.
14. H. Krawczyk, P. Eronen. *HKDF*. RFC 5869, 2010.
15. Enunciado del Proyecto 1 — ST0263/SI3007 «DFSha», 2026-2.

Anexo A. Reproducibilidad

```
git clone <repositorio> && cd SIST-DISTR
uv sync --extra dev --extra report
uv run dfsha-cluster up --control 3 --data 5 --zones 2
eval "$(uv run dfsha-cluster env)" && uv run dfsha login -u admin -p dfsha-admin-dev
uv run pytest                          # las 175 pruebas (≈6 min)
uv run python bench/bench.py all       # benchmarks -> bench/results/*.json
uv run python report/figs.py && uv run python report/build_report.py
```